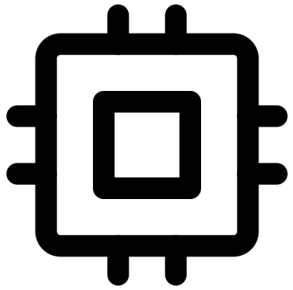
# MANGO BLOCK 入門

用 Python 控制你的開發板

Codpy Section



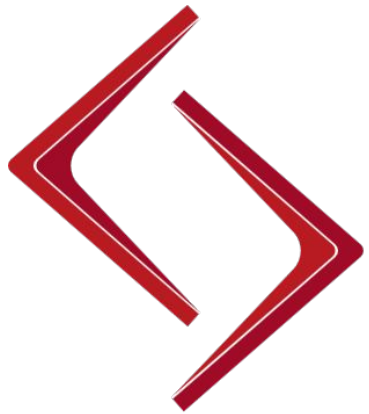
上午講者 : 蘇立原

下午講者 : 鄭元凱

# SPONSOR



# 國立成功大學資訊工程學系暨研究所



敏求智慧運算學院

Miin Wu School of Computing




社團法人中華民國愛自造者學習協會



NOVATEK

# SPONSOR

國立成功大學資訊工程學系暨研究所

敏求智慧運算學院
Miin Wu School of Computing

PROGRAM the WORLD
社團法人中華民國愛自造者學習協會

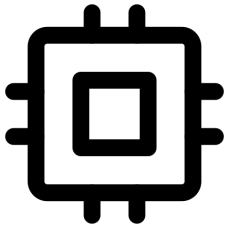
NOVATEK

# 今日流程

| | | |
|---|---|---|
| 09:00 | 開場與設備介紹 | |
| 09:20 | 環境安裝 | 實作 |
| 10:00 | LED 與按鈕 | 實作 |
| 10:30 | 中場休息 | |
| 10:40 | 蜂鳴器 | 實作 |
| 11:00 | 網頁儀表板實作 | 實作 |
| 11:45 | 展示與總結 | |
| 13:00 | 舵機控制與通訊架構 | 實作 |
| 14:30 | 多舵機操控 | 實作 |

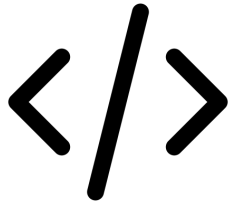
課程手冊

# 今天的目標



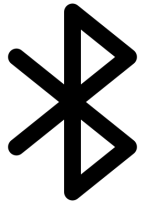
## 認識板子

彩燈、蜂鳴器、按鍵
在板子上的哪裡



## 用程式控制它

輸出:燈、聲音
輸入:按鈕



## 用網頁遙控它

藍牙網頁儀表板
中午前完成

# 先看成品


全部彩燈 on_led
選顏色 → 按「套用」，6 顆燈一起變色。
套用
全關
單顆彩燈 on_led
先選顏色，再點一顆圓點，只有那一顆燈會變色。
燈光效果 on_effect
呼吸、跑馬燈會使用目前設定的顏色。
靜態
彩虹
呼吸
跑馬燈
警示燈
亮度 on_brightness
16 %
蜂鳴器 tone_on / tone_off
按琴鍵，板子的蜂鳴器發出聲音。
Do
Re
Mi
Fa
Sol
La
Si
Do+
按鍵狀態 read_keys
按下板子上的 Key1 / Key2，網頁上對應的方塊要亮起來。
KEY1
KEY2
小遊戲：反應力 on_game_start / game_result
按「開始」後板子亮紅燈，等燈變綠就按 Key1，越快越好；還沒變綠就按算偷跑。（要先完成 TODO 7）
開始
反應時間：264 ms

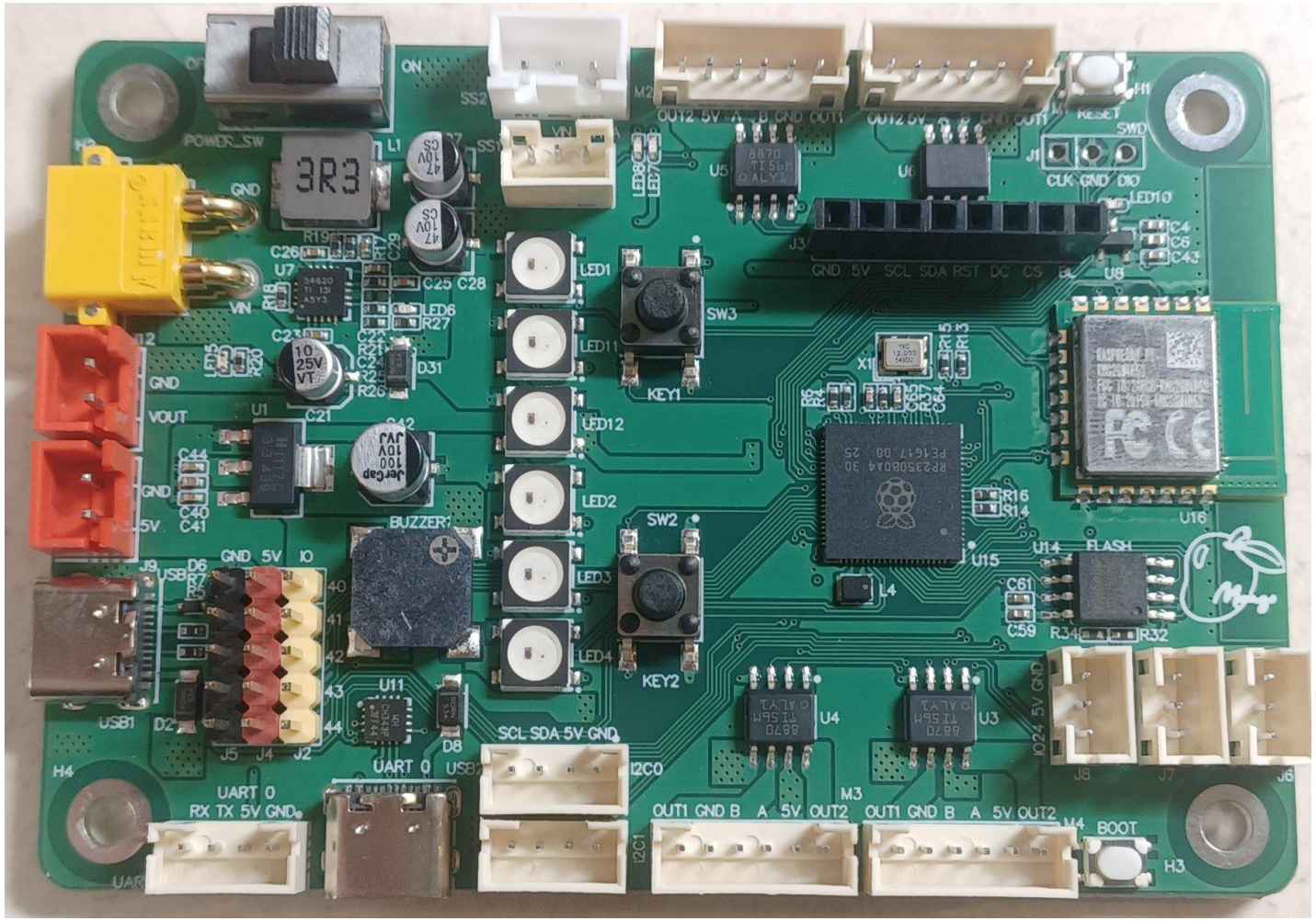
Part 0

# 認識 MANGO BLOCK

板子上有什麼

09:05 – 09:20

# 這就是 MANGO BLOCK




## MANGO BLOCK

動力驅動板

一塊板子＝一台小電腦

可以跑 Python（MicroPython）

可以用藍牙、Wi-Fi

可以控制舵機（下午）

# 兩顆主要的晶片

## RP2350B

**大腦**

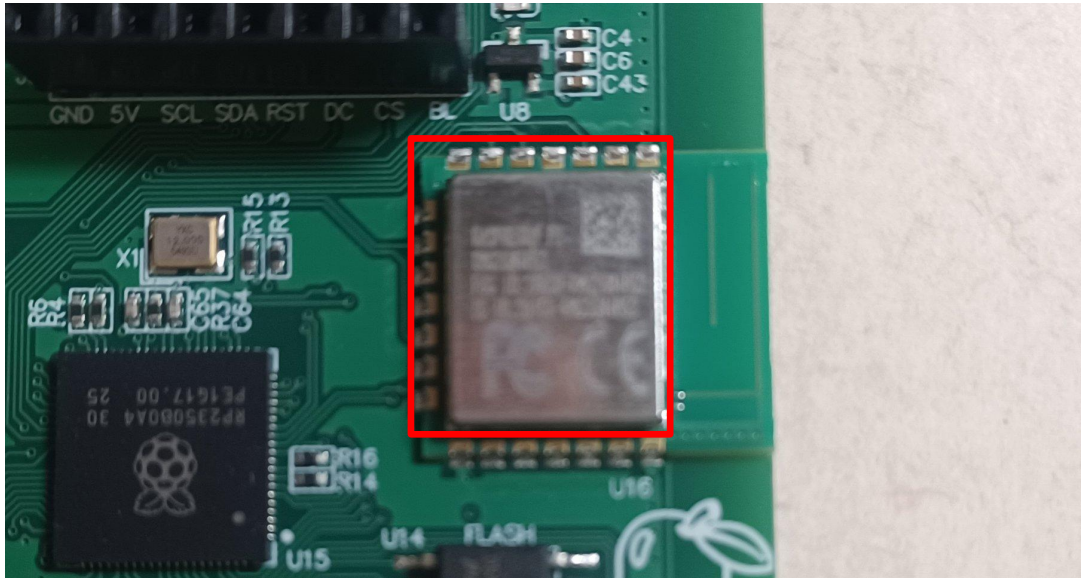
執行你寫的程式
控制燈、蜂鳴器、按鍵、舵機



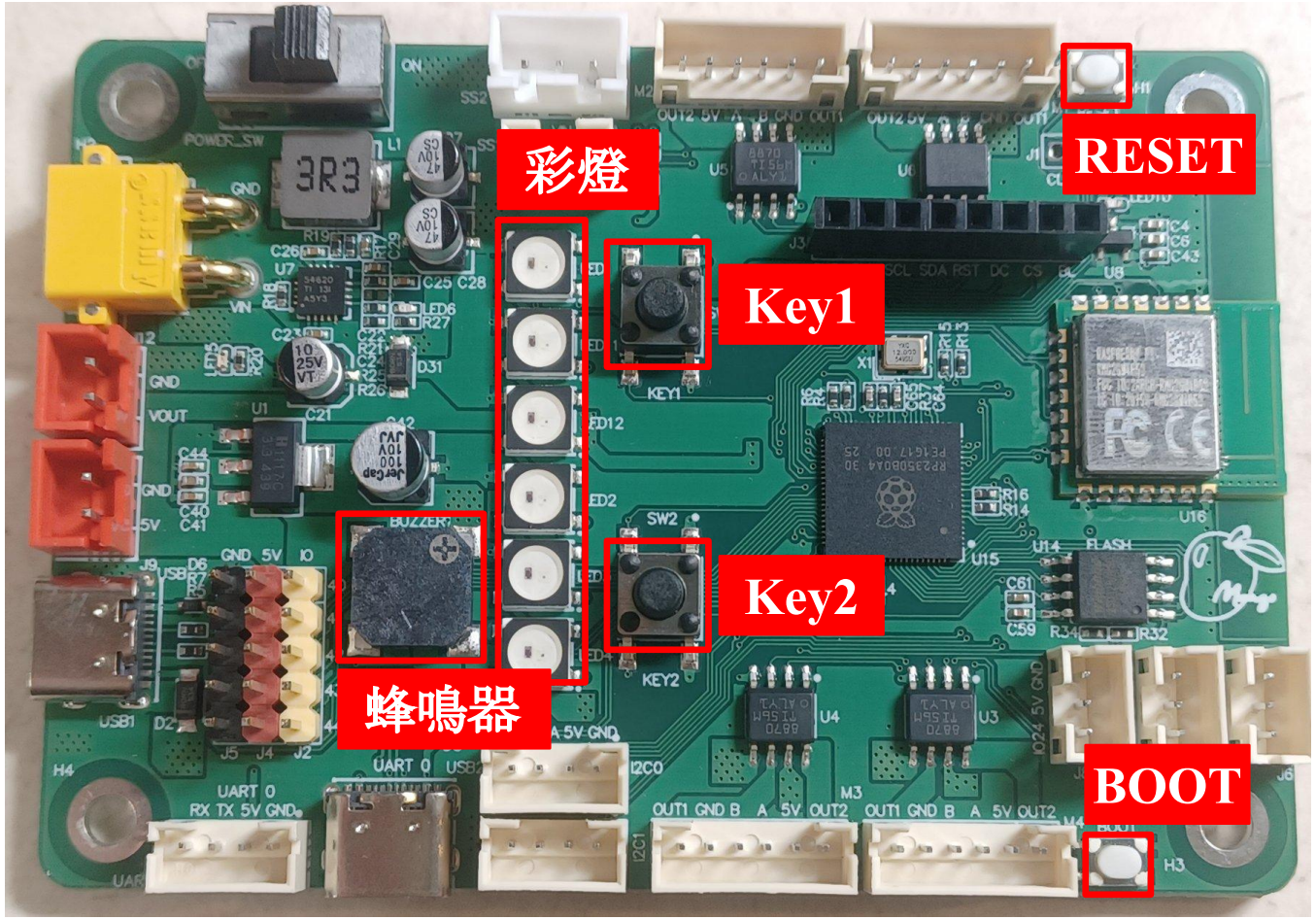

## RM2

**嘴巴和耳朵**

藍牙、Wi-Fi 無線通訊
今天的專題靠它和網頁連線



# 今天會用到的元件




**6 顆彩燈**

WS2812B，可以變任何顏色

**蜂鳴器**

發出不同音高

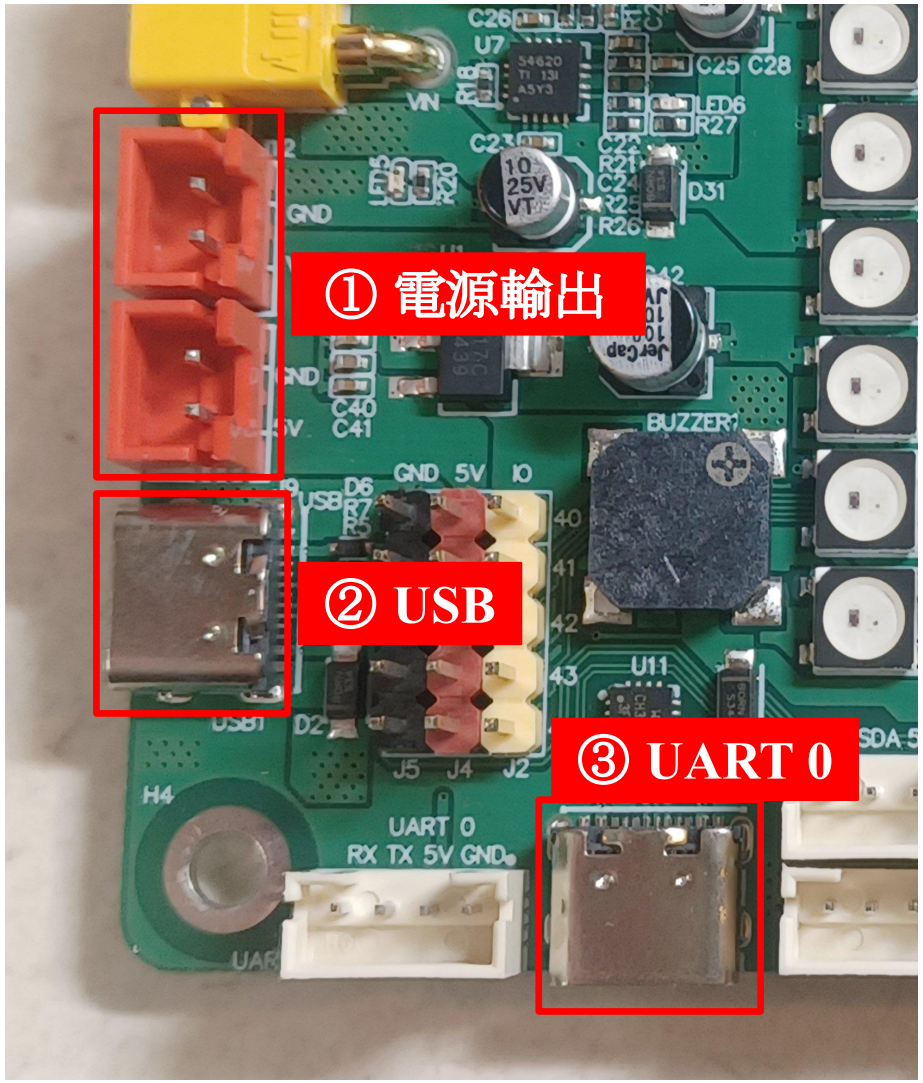
**Key1 / Key2**

兩顆給你用的按鍵

**BOOT / RESET**

燒錄韌體、重新開機

# 接電腦：只能用這個 Type-C




**① 紅色接頭：電源輸出**

GND / VOUT / 5V，把電送給其他模組用
不是接電腦的地方

**② USB（Type-C）：接電腦**

供電 + 上傳程式
今天全部都用這個孔

**③ UART 0（Type-C）：不能通用**

傳輸方式不一樣，接上電腦
也沒辦法上傳程式

**連接電腦：一定要用 ② USB**

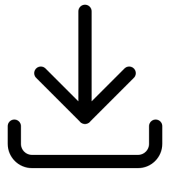
Part 1

# 環境設定

下載 · 燒錄韌體 · Python · VS Code · MicroPico

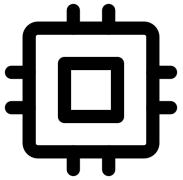
09:20 – 10:00

# 要準備的東西



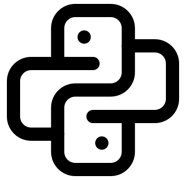
**課程檔案**

GitHub →
Download ZIP



**課程韌體**

課程版
.uf2 檔



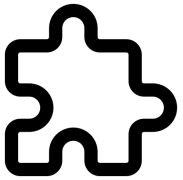
**Python**

儀表板實作
開網頁用



**VS Code**

寫程式的地方



**MicroPico**

VS Code 擴充功能
連到板子

# 下載課程檔案

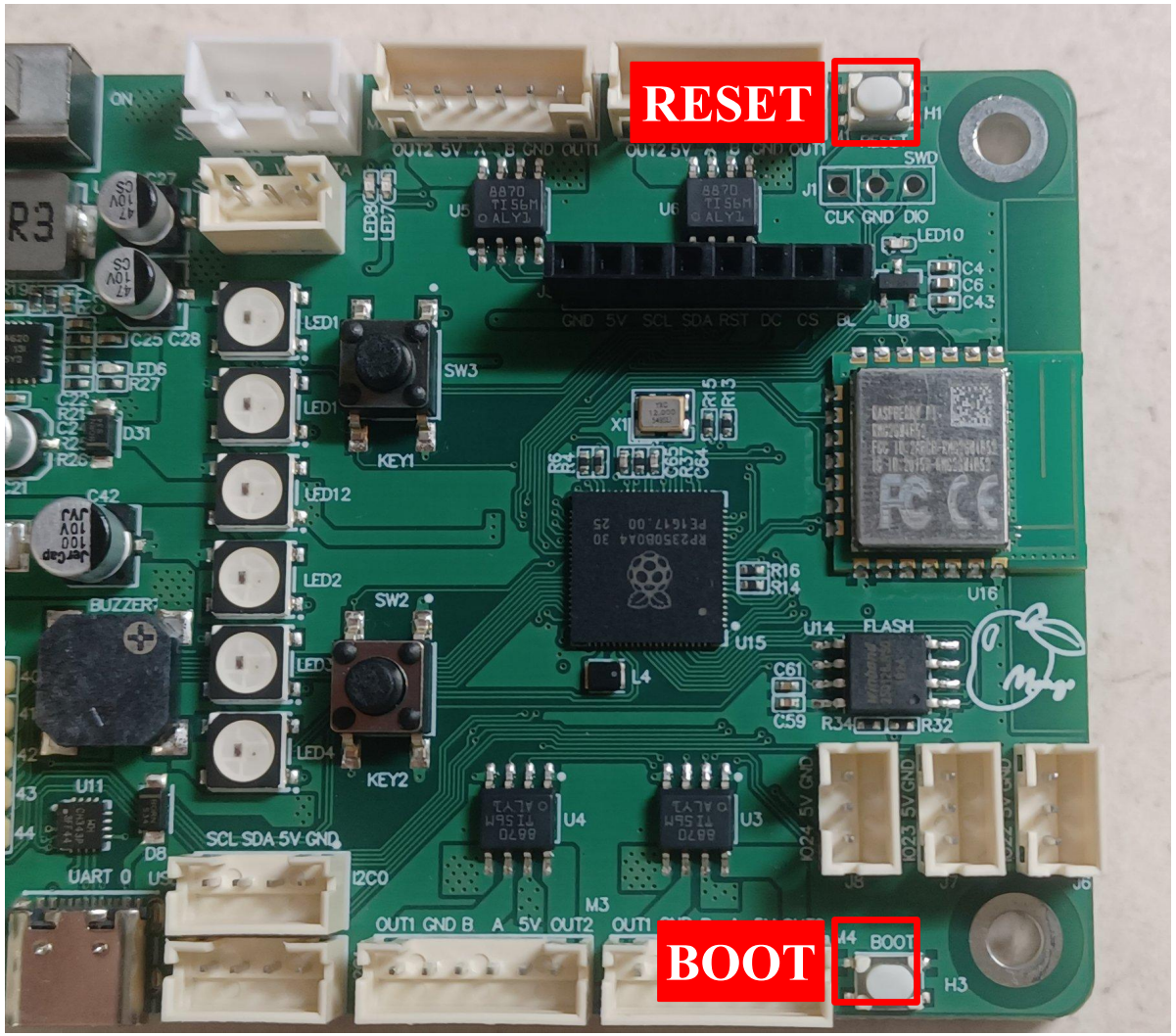



1

## Code → Download ZIP

解壓縮到桌面
資料夾裡有 morning、firmware

# 刷韌體




1. **按住 BOOT，按一下 RESET**
   **再放開 BOOT**

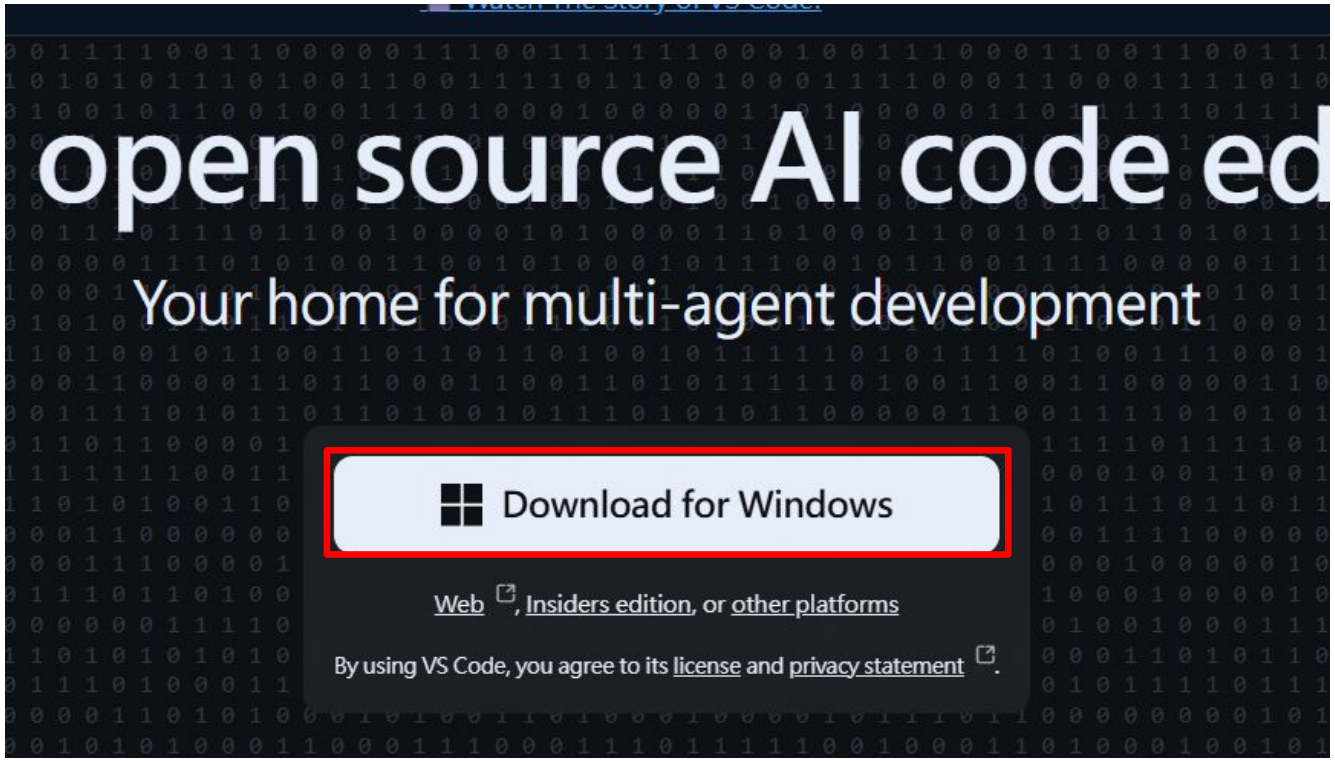
   電腦會出現 RP2350 磁碟

2. **把韌體拖進 RP2350 磁碟**

   ```
   firmware/MANGO_X4-v1.29.0.uf2
   ```

3. **磁碟自動消失 = 完成**

# 安裝 VS Code 與 MicroPico

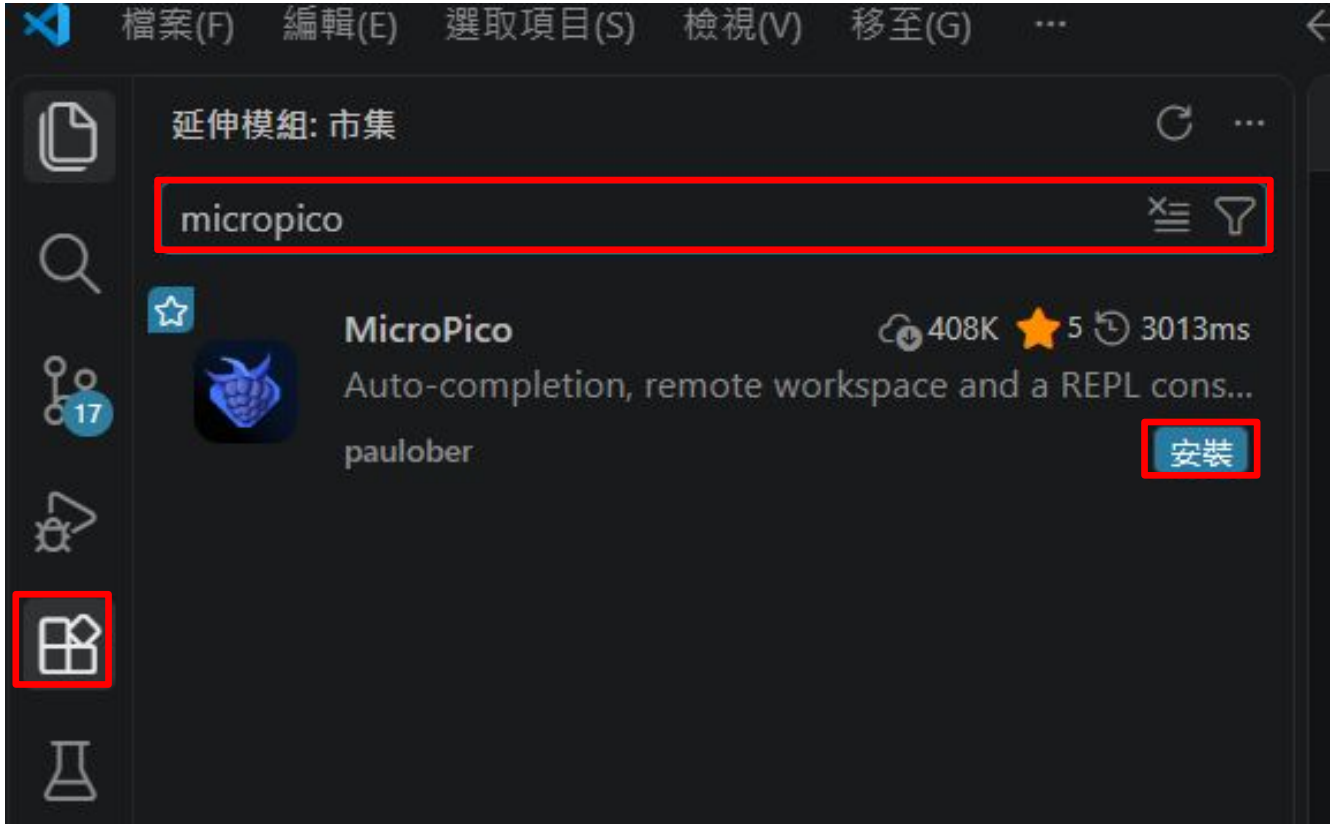



1

## 安裝 VS Code

code.visualstudio.com

# 安裝 VS Code 與 MicroPico




**2**

## 搜尋 MicroPico
## 按安裝

點左邊直排的「擴充功能」圖示（四個方塊，或按 Ctrl+Shift+X）
它建議的 Python 擴充功能也一起裝

# 安裝 VS Code 與 MicroPico



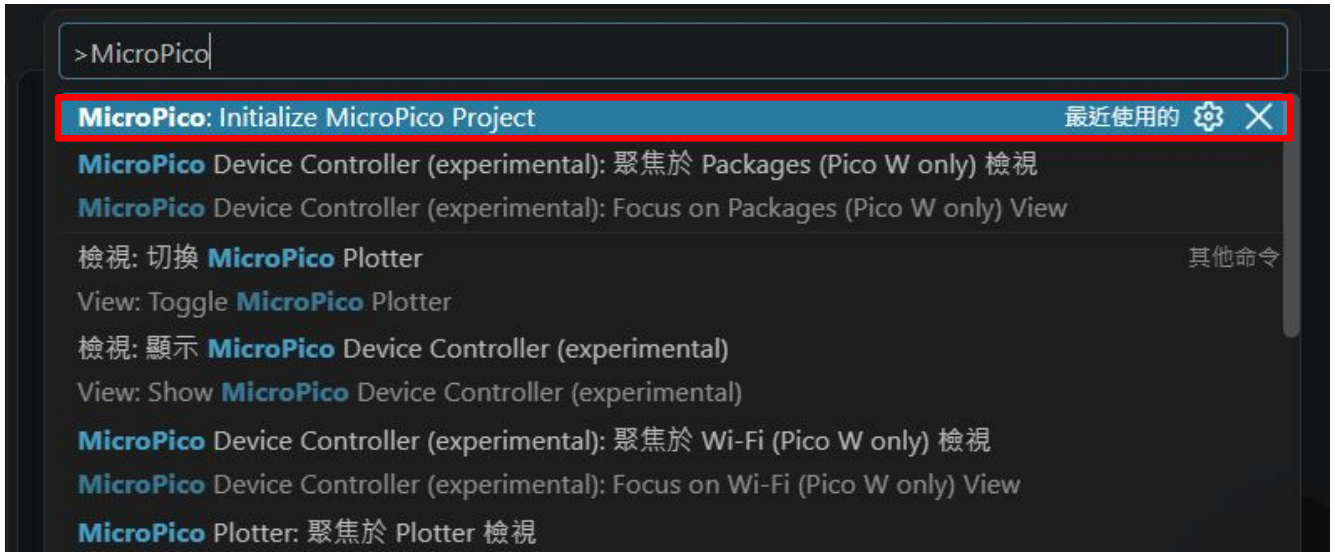

3

## 安裝 Python

Python 官網, 滑鼠停在 Downloads
找到「Or get the standalone installer for」→ 按 Python 3.15.0
下載 .exe 後打開

**一定要勾 Add python.exe to PATH**

# 安裝 VS Code 與 MicroPico



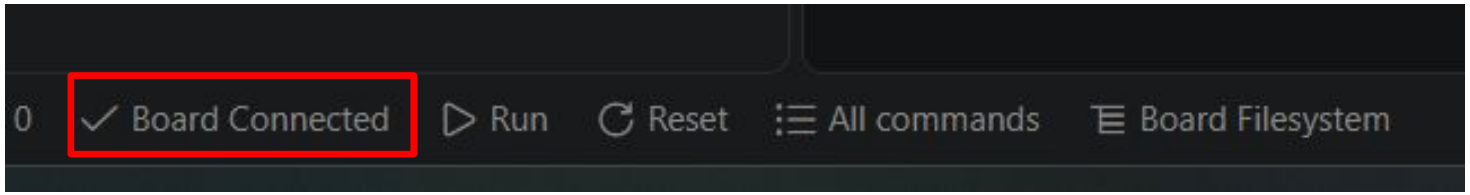

4

## 開資料夾
## 初始化專案

檔案 → 開啟資料夾 → 課程資料夾
Ctrl+Shift+P → 輸入 MicroPico

**MicroPico: Initialize MicroPico Project**

# 連上了嗎？




左下角顯示

**Board Connected**

就是連上了

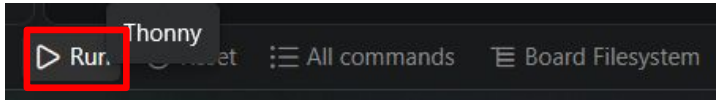
**一直 Disconnected？**

**首先試看看 拔掉再插**

接的是 ② USB 嗎？換看看別條同樣能傳資料的線？

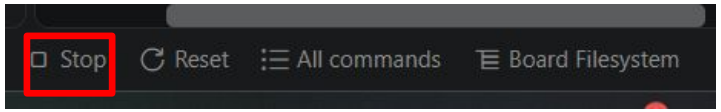
# 三個動作

## Run




把打開的檔案送到板子執行
不會存到板子

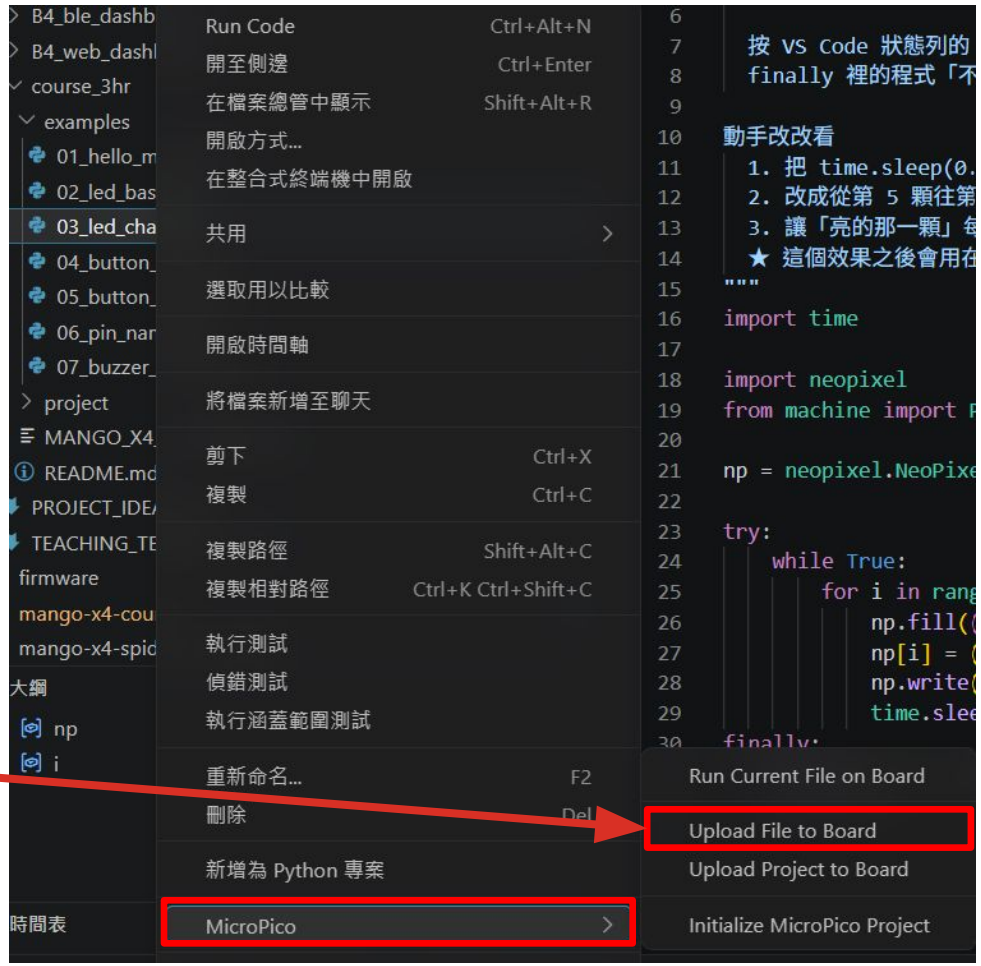
## Stop




停止有 while True 的程式
也可以在終端機按 Ctrl+C

## Upload

檔案按右鍵 → MicroPico
→ Upload File to Board



實作 01

# 板子點名

```
morning/examples/01_hello_mango.py
```

**動手改改看**

1. 打開檔案 → 按 Run
2. 看 6 顆燈亮起、嗶一聲
3. 已完成的同學：改改看燈的顏色

**檢查點**　終端機出現「環境 OK！」

`01_hello_mango.py` 的開頭匯入

```
import time

import neopixel
from machine import PWM, Pin
```

開頭這幾行叫「匯入模組」：
現在先跑跑看，實作 02 之前會介紹。

**實作 01**

# 20 分鐘

燈亮了、嗶了、看到「環境 OK！」

Part 2

# LED 與按鈕

模組 · pin 腳 · 輸出 · 輸入

10:00 – 10:30

# 程式的開頭:匯入模組

`01_hello_mango.py` 的開頭

```
import time

import neopixel
from machine import PWM, Pin
```

Python 本身只會基本的事。
要控制板子, 先把工具匯入進來



**`time`**

時間:time.sleep(0.1) 等 0.1 秒



**`neopixel`**

彩燈:控制 6 顆 WS2812B 彩燈



**`machine 的 Pin`**

腳位:用名字找到板子上的元件



**`machine 的 PWM`**

蜂鳴器:控制音高和音量

# 今天早上會用到的 腳位名稱與實際編號

| 名字 | GPIO | 接到 |
| --- | --- | --- |
| WS2812B | 45 | 6 顆彩燈 |
| BUZZER | 46 | 蜂鳴器 |
| KEY1 | 30 | 按鍵 1 |
| KEY2 | 31 | 按鍵 2 |

**GPIO = 晶片上可以自己控制的 腳**

# 名字和號碼是同一支 腳

```
Pin("KEY1")
```

好讀、不容易寫錯

＝

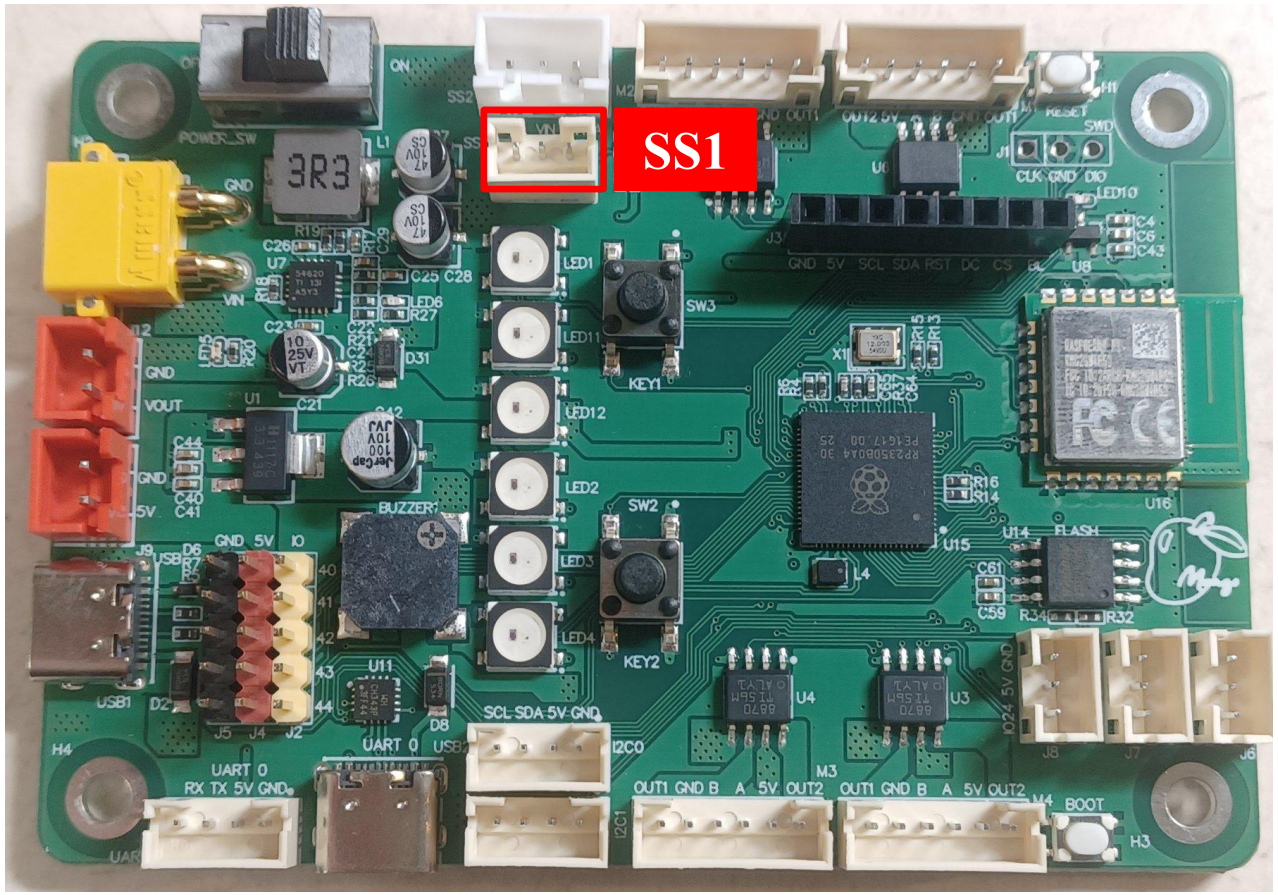
```
Pin(30)
```

要背號碼

**課程韌體幫每支 腳取了名字, 上午程式裡一律用名字**

板子上所有腳位：morning/PINS.md

# 下午會用到的接頭




## SS1

**接 Hiwonder 匯流排舵機**

| 名字 | GPIO | 用途 |
|---|---|---|
| SS_TX | 8 | 送封包給舵機 |
| SS_RX | 9 | 讀回舵機的回覆 |

一條線串很多顆，用封包 + ID 指定誰
UART · 115200 baud

# 程式報紅字時怎麼看

終端機

```
Traceback (most recent call last):
  File "<stdin>", line 12, in <module>
NameError: name 'led' isn't defined
```

**① 看Error：錯誤種類**

NameError = 名字打錯了
（led 應該是 leds）

**② 看 line 12：第幾行**

在 VS Code 找到第 12 行

| 最後一行寫著 | 通常是 |
| --- | --- |
| SyntaxError | 打到全形符號（，：（）“”）或少了冒號、括號 |
| NameError | 名字拼錯，或用了還沒建立的變數 |
| IndentationError | 空白（縮排）沒對齊 |
| TypeError | 數字的種類不對，例如顏色用到小數，要 int() |

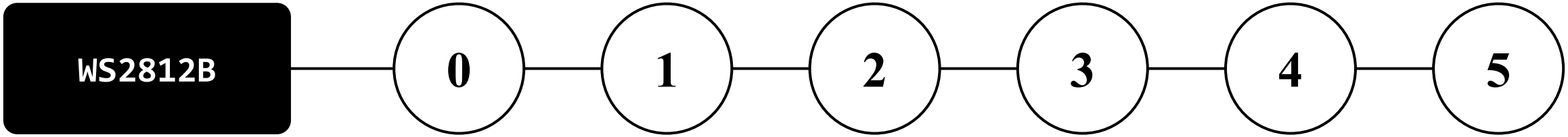
# 用名字把元件叫出來

| 程式 | 之後用這個名字 | 會用在 |
| --- | --- | --- |
| `leds = neopixel.NeoPixel(Pin("WS2812B"), 6)` | → **`leds`**<br>6 顆彩燈 | 02、TODO 1、4 |
| `buzzer = PWM(Pin("BUZZER"))` | → **`buzzer`**<br>蜂鳴器 | 04、TODO 3 |
| `button1 = Pin("KEY1", Pin.IN, Pin.PULL_DOWN)` | → **`button1`**<br>按鍵 1 | 03、TODO 2 |

**Pin("名字") 找到那支腳，再交給 NeoPixel / PWM，變成可以控制的東西**

實作 02

# 6 顆燈, 接在同一支 腳上




編號從 0 開始: 0 ~ 5

```
leds = neopixel.NeoPixel(Pin("WS2812B"), 6)
```

實作 02

# 先記下來，write() 才送出

```
leds[0] = RED
```

→

```
leds.write()
```

只是先記下來（RED = (30, 0, 0)）
燈還不會變

這一行才真的把顏色
送到燈上

**沒有 write()，燈就不會變**

實作 02

# 用 for 迴圈控制 6 顆

02_led.py

```
for number in range(6):
    leds[number] = BLUE
leds.write()
```

**number 會依序是**

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|

6 顆都設定完, 最後 write() 一次

**全部同一色也可以寫**

```
leds.fill(BLUE)
```

實作 02

# 跑馬燈:一格一格畫

02_led.py

```
# ③ 跑馬燈(按 Stop 停止)
while True:
    for number in range(6):
        leds.fill(OFF)
        leds[number] = GREEN
        leds.write()
        time.sleep(0.1)
```



**全部關掉**

`leds.fill(OFF)`



**點亮第 number 顆**

`leds[number] = GREEN`



**送出**

`leds.write()`



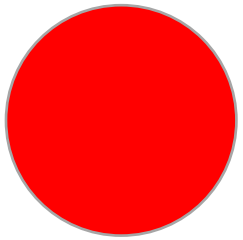
**等一下，換下一顆**

`time.sleep(0.1)`

動畫 = 很快地一張一張換圖
檔案最後的 finally:按 Stop 時也會關燈

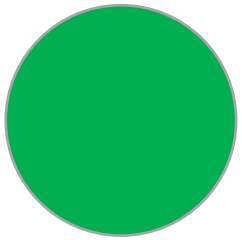
實作 02

# 顏色 = (紅, 綠, 藍)



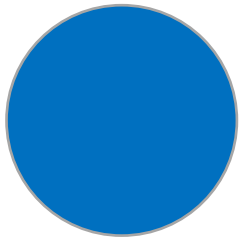
紅

(30, 0, 0)



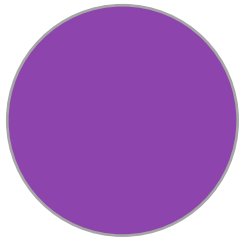
綠

(0, 30, 0)



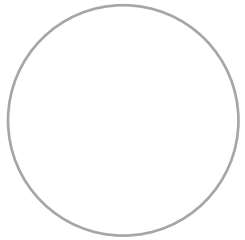
藍

(0, 0, 30)



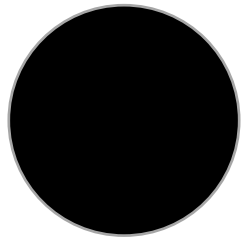
紫

(20, 0, 20)



白

(30, 30, 30)



關

(0, 0, 0)

**0 ~ 255**

每個數字的範圍

**30**

就夠亮了, 255 太刺眼

實作 02

# 控制彩燈

```
morning/examples/02_led.py
```

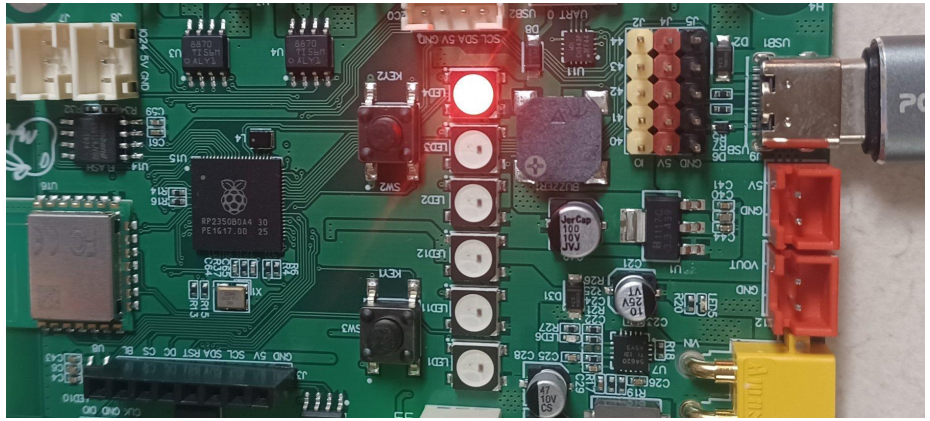
① 一顆 → ② 全部（for 迴圈）→ ③ 跑馬燈

**動手改改看**

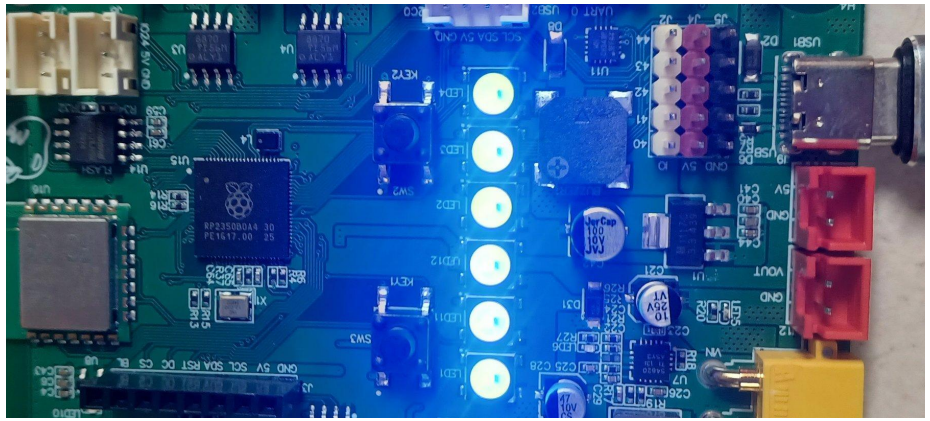
1. 把 RED 改成 (0, 0, 30)，第 0 顆變什麼色？
2. 跑馬燈的 sleep 改成 0.3 或 0.03
3. 做一個新顏色 PURPLE，讓跑馬燈用它

**檢查點** 能點亮一顆、設定全部，做出自己的跑馬燈

★ 會用在儀表板的 TODO 1 和 TODO 4



一顆：第 0 顆亮紅



全部：6 顆亮藍

實作 02

# 10 分鐘

實作 03

# 讀按鈕：一行設定，一行讀取

## 設定

```
button1 = Pin("KEY1", Pin.IN, Pin.PULL_DOWN)
```

↑ 哪一支腳　↑ 輸入　↑ 沒按時固定是 0

## 讀取

```
button1.value()
```

→ **1** 按下　**0** 放開

實作 03

# 輸出 OUTPUT、輸入 INPUT

## 輸出

**OUTPUT**

程式 → 外面

螢幕、喇叭、電燈

彩燈、蜂鳴器

```
Pin("名字", Pin.OUT)
```

## 輸入

**INPUT**

外面 → 程式

鍵盤、滑鼠、感測器

Key1、Key2 按鍵

```
Pin("名字", Pin.IN)
```

**02、04 是輸出:程式「送出去」; 03 開始學輸入:程式「讀進來」**

實作 03

# 為什麼要 PULL_DOWN？

## 大部分板子：沒加

沒按的時候，腳位沒接到任何東西
讀到的數字會亂飄

0 1 1 0 1 0 0 1



沒有按下

## 加 Pin.PULL_DOWN

沒按的時候，把腳位「拉」到 0
數值才會穩定

0 0 0 0 1 1 1 1

按下按鈕

MANGO BLOCK 板子上已經有下拉電阻，所以沒加也不會亂跳

**但不是每一塊控制板都有 → 養成習慣：讀按鍵記得加 Pin.PULL_DOWN**

實作 03

# 一直讀，一直判斷

03_button.py

```
while True:
    button1_now = button1.value()
    if button1_now == 1:
        leds.fill(RED)
    else:
        leds.fill(OFF)
    leds.write()
    time.sleep(0.02)
```



**讀按鈕**

value() → 0 或 1



**判斷**

1 亮紅燈，0 關燈



**送出**

leds.write()



**等一下，重複**

每 0.02 秒讀一次

**程式跑得很快，看起來就像「按住才亮」**

實作 03

# 按鍵的兩種用法



## 按住才亮

像門鈴：按著才響，放開就停

**按鍵 1：只看「現在」是 0 還是 1**

```
if button1_now == 1:
    leds.fill(RED)
```



## 按一下開、再按一下關

像房間的電燈開關：
按一下亮，再按一下關

**按鍵 2：要記住「上一次」**

```
light_is_on = not light_is_on
```

**03_button.py 兩種都有：按鍵 1 按住才亮、按鍵 2 按一下切換**

實作 03

# 按一下切換：抓住「剛剛被按下」

讀到的值

| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|

↑ 上一次 0、這一次 1 → 剛剛被按下！

按住不放的那幾個 1 不算：只有「從 0 變成 1」的那一瞬間才切換

03_button.py

```
if button2_before == 0 and button2_now == 1:
    light_is_on = not light_is_on       # 開 ↔ 關 互換
button2_before = button2_now            # 記住這一次，下一圈拿來比
```

實作 03

# 把 time.sleep 拿掉會怎樣？

03_button.py

```
while True:
    ...讀按鍵、決定顏色...
    leds.write()
    # time.sleep(0.02)
```

按鍵 2 還能切換

**按住按鍵 1 卻不亮**



**彩燈要「停頓一下下」才會換顏色**

收到資料後，要停一小段時間，
才會把新顏色顯示出來



**沒有 sleep：write() 一個接一個**

迴圈跑太快，彩燈一直在收資料，沒有空檔顯示 → 看起來沒反應

那為什麼按鍵 2 還可以？ → 下一頁

實作 03

# 為什麼按鍵 2 還可以？

## 按鍵 2 切換時會 print

印字要花時間 → 剛好給了彩燈「停頓一下下」的空檔 → 新顏色顯示出來

**試試看**

sleep 一樣拿掉，在每一個 if 底下都加一行 print

```
if button1_now == 1:
    print("按鍵 1")
```

→ **按鍵 1 也會亮了**

## 結論：迴圈裡記得留一點等待

`time.sleep(0.02)`：彩燈來得及換顏色，也避免按鍵抖動被算成好幾次

實作 03

# 按住才亮、按一下切換

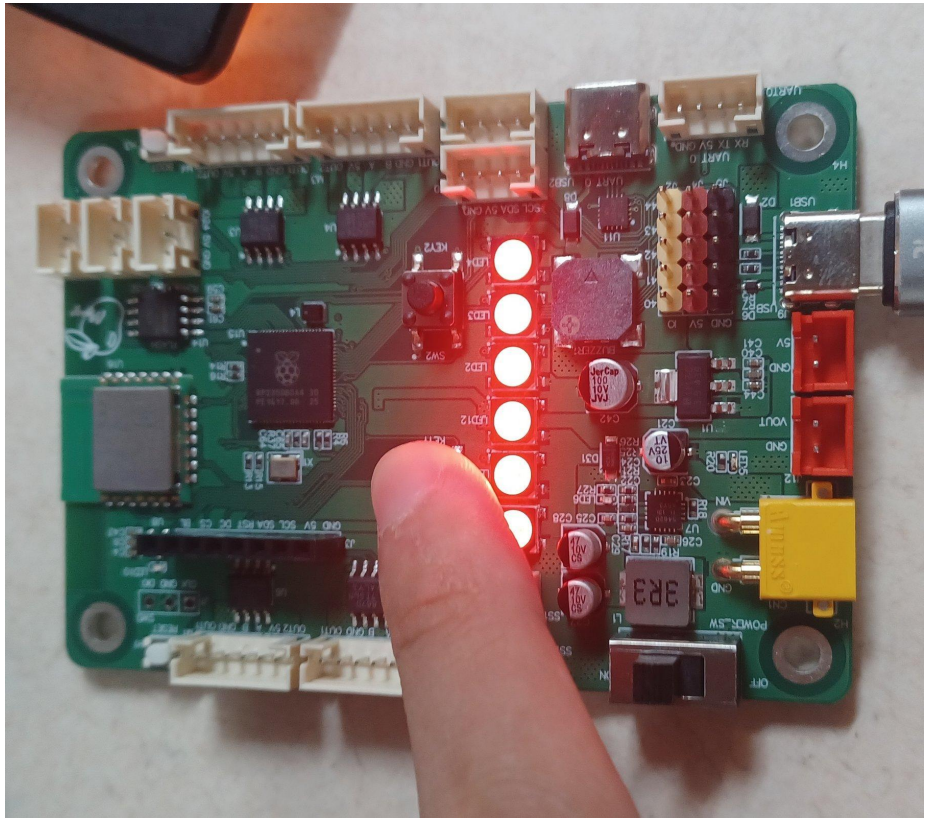
```
morning/examples/03_button.py
```

按鍵 1：看「現在」
按鍵 2：抓「上一次 0、這一次 1」

**動手改改看**

1. 按住按鍵 1 改成亮藍燈 (0, 0, 30)
2. 加 print(button2_now)，按一下印出幾個 1？

**檢查點** 按鍵 1 按住才亮、按鍵 2 能切換開關



實作 03

# 10 分鐘

# 中場休息

10:30 – 10:40

Part 3

# 蜂鳴器

音高 · 音量 · 播一首歌

10:40 – 11:00

實作 04

# 其實 01 已經嗶過了

01_hello_mango.py

```
buzzer = PWM(Pin("BUZZER"))

buzzer.freq(880)          # 音高
buzzer.duty_u16(3000)     # 音量(開始發聲)
time.sleep(0.2)           # 響 0.2 秒
buzzer.duty_u16(0)        # 音量 0 = 停止
```

1. **發聲**
   `freq + duty_u16`
2. **等一下**
   `sleep`
3. **停止**
   `duty_u16(0)`

**蜂鳴器的用法就這 4 行，打開 01 找找看**

沒有 duty_u16(0) → 程式停了還一直響

實作 04

# 蜂鳴器只要記兩個 function

## `freq()`

### 頻率 = 音高

數字越大聲音越高
440 = La

## `duty_u16()`

### 音量

3000 左右就夠大聲, 0 = 安靜
最大約 32768, 再大反而變小聲

實作 04

# play()：把「嗶一聲」包成一個指令

04_buzzer_melody.py

```
def play(pitch, seconds):
    if pitch != REST:
        buzzer.freq(pitch)
        buzzer.duty_u16(VOLUME)
    time.sleep(seconds * 0.8)
    buzzer.duty_u16(0)
    time.sleep(seconds * 0.2)
```

```
play(DO, 0.4)    # 播 Do，0.4 秒
```

1. **pitch、seconds**

   音高、這個音有幾秒

2. **pitch != REST**

   不是休止符才發聲（REST = 0）

3. **響 80%、安靜 20%**

   兩個一樣的音才分得開

**寫好一次，之後一行就能播一個音**

實作 04

# 歌譜是資料，程式只負責播

資料：歌譜

```
song = [
    (DO, 0.4), (DO, 0.4),
    (SOL, 0.4), (SOL, 0.4),
    (LA, 0.4), (LA, 0.4), (SOL, 0.8),
]
```

程式：一個一個播

```
for pitch, seconds in song:
    play(pitch, seconds)
```

**（音高，秒數）**

音高 0 = 休止符

**為什麼要安靜 20%？**

兩個一樣的音才分得開：Do Do 不會黏成一個長 Do

**換一首歌 = 只改資料，不用改程式**

實作 04

# 音高對照

| Do | Re | Mi | Fa | Sol | La | Si | Do⁺ |
|---|---|---|---|---|---|---|---|
| 262 | 294 | 330 | 349 | 392 | 440 | 494 | 523 |

**網頁儀表板上的琴鍵, 送來的就是這些數字**

實作 04

# 老師示範:換一首歌

teacher/rickroll.py

```
song = [
    (C, S), (D, S), (F, S), (D, S),
    (A, Q), (A, Q), (G, Q * 1.5),
    ...
]
```

**Never Gonna Give You Up**

play() 一行都沒改
只換了 song 這份歌譜

**加碼:每個音配一個顏色, 燈會跟著音樂閃**

**換你:把你喜歡的歌改成 (音高, 秒數)**

實作 04

# 小星星

```
morning/examples/04_buzzer_melody.py
```

**動手改改看**

1. 補完後半段：Fa Fa Mi Mi Re Re Do
2. 換成你自己會的一首歌

**檢查點** 能播放自己改過的旋律



實作 04

# 10 分鐘

Part 4

# 網頁儀表板實作

網頁已完成, 你寫板子端

11:00 – 11:55

# 今天要做的系統



網頁儀表板 → 藍牙 → mango_ble.py → dashboard.py → 燈 · 蜂鳴器 · 按鍵

已完成 | 你寫的 | 01 ~ 04 學過的

藍牙的原理今天不講, 交給工具庫 mango_ble.py

# 網頁按一下 → 你的函式被呼叫

整個實作只要記住這一句

# 實作 01~04 和網頁儀表板，哪裡不一樣？

## 實作 01~04：程式自己跑

- 按 Run，從第一行跑到最後一行
- while True + sleep 自己控制節奏
- 自己 leds.write() 送出

```
while True:
    leds.fill(OFF)
    ...
    leds.write()
    time.sleep(0.1)
```

## 儀表板：你寫函式，網頁來呼叫

- 網頁按下按鈕 → 呼叫你的函式
- 函式只做「一件小事」就結束
- 不寫 while True、sleep、write

```
def on_led(led_number,
           red, green, blue):
    # 網頁選了顏色才會跑到這裡
```

**裡面要寫的東西，還是 01~04 學過的那幾行**

# 同一件事，兩種寫法

**實作 01~04（自己跑）**

```
buzzer.freq(880)
buzzer.duty_u16(3000)
time.sleep(0.2)
buzzer.duty_u16(0)
```

→

**儀表板（網頁呼叫）**

```
def tone_on(pitch):
    buzzer.freq(pitch) ...
def tone_off():
    buzzer.duty_u16(0)
```

**01 嗶一聲 → TODO 3**

```
button1_now = button1.value()
```

→

```
def read_keys():
    return 按鍵1, 按鍵2
```

**03 讀按鍵 → TODO 2**

**把「做一次的那幾行」搬進函式；sleep 拿掉，交給工具庫**

# 為什麼函式裡不能寫 sleep？

**工具庫一個人要同時顧三件事：**



**顧藍牙**

隨時接收網頁
送來的資料



**顧按鍵**

每 0.01 秒
問一次 read_keys



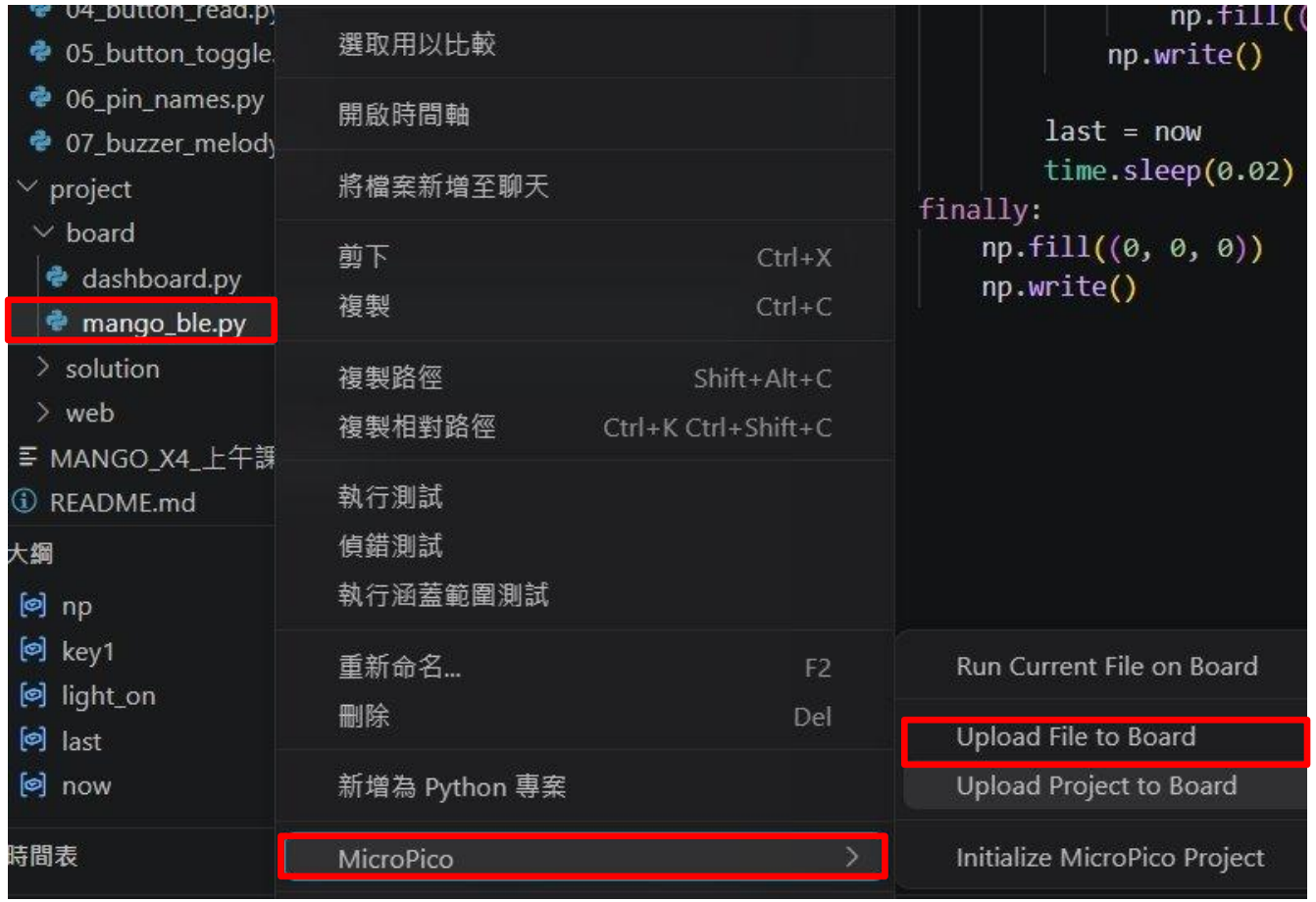
**顧燈光**

每 0.03 秒
呼叫一次 tick

**你的函式裡寫了 time.sleep(1) → 這 1 秒工具庫什麼都不能做**
**→ 按鍵沒反應、動畫停住、網頁斷線**

終端機出現「⚠ … 執行太久」就是在提醒你這件事

# 準備步驟

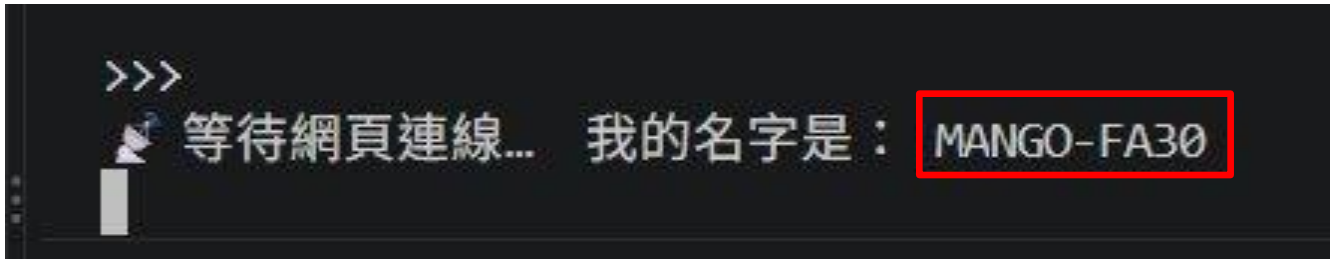



**1**

## 上傳工具庫

mango_ble.py 按右鍵
→ MicroPico
→ Upload File to Board
只要做一次

# 準備步驟




2

## Run dashboard.py
## 記下自己的名字

教室裡有 30 塊板子
要連到自己的那塊

```
MANGO-XXXX
```

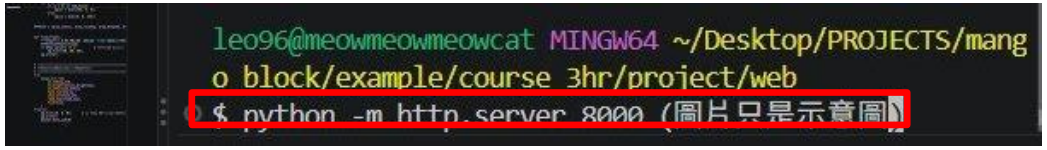
# 準備步驟

## 3 開網頁伺服器




### ① 開一個新的終端機

右上角「+」→ 新增終端機
(PowerShell)

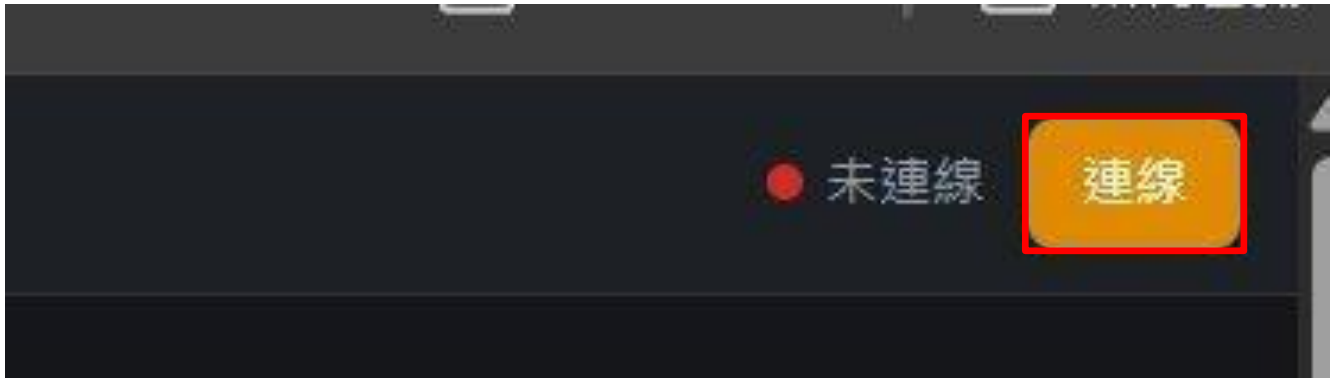



### ② 輸入兩行指令，每行按 Enter

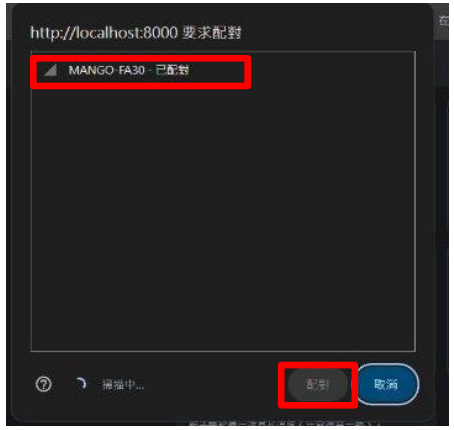
```
cd morning/project/web
python -m http.server 8000
```

**要在 PowerShell 分頁打，不是 MicroPico vREPL**
**找不到 python？改用 py -m http.server 8000**

# 準備步驟




① 按右上角「連線」




② 選自己的 MANGO-XXXX → 配對

4

## 連到自己的板子

Chrome / Edge 開
localhost:8000
→ 連線 → 選 MANGO-XXXX

**檢查點：網頁顯示「已連線」**

改完程式：Stop → Run → 網頁重新連線

# 先不用網頁：自己呼叫你的函式

dashboard.py 最下面

```
TEST_WITHOUT_WEB = True
```

改成 True → 按 Run：和實作 01~04 一樣，跑完就看到結果

```
on_led(255, 255, 0, 0)   # 假裝選了紅色
run_frames(1)            # 假裝畫 1 秒
tone_on(440)             # 假裝按了 La
read_keys()              # 印出按鍵
on_effect(3)             # 假裝按跑馬燈
```

**1 自己呼叫**

TEST_WITHOUT_WEB = True
確認每個 TODO 有反應

**2 換網頁呼叫**

改回 False → Run
網頁按鈕呼叫同一批函式

**函式就是「可以被呼叫的東西」：誰呼叫都一樣**

# TODO 1：on_led

**必做**　**和 02 的 for 迴圈一樣**

網頁選了顏色：led_number 0 ~ 5 只改那一顆、255 改全部

```
def on_led(led_number, red, green, blue):
    # 虛擬碼：
    # 如果 led_number 是 255：
    #     對每一顆 number：
    #         led_colors[number] = 顏色
    # 否則：
    #     led_colors[led_number] = 顏色
    # 顏色 = (red, green, blue)
```

**測試**　先按「靜態」，再選顏色按套用、點單顆圓點

### 和 02 比一比

```
for number in range(6):
    leds[number] = BLUE
```

leds 換成 led_colors
BLUE 換成網頁送來的顏色

不用 write()：tick 會畫到燈上

# TODO 2：read_keys

**必做** **和 03 讀按鍵一樣**

回傳兩顆按鍵的狀態（要回傳兩個值）

```
def read_keys():
    # 虛擬碼：
    # key1 = 讀 button1 的值
    # key2 = 讀 button2 的值
    # 回傳 key1, key2
    return 0, 0  # ← 改這行
```

**測試** 按住板子 Key1 / Key2，網頁方塊亮起

**和 03 比一比**

```
button1.value()
```

button1、button2 已經幫你設定好了

回傳兩個值的寫法：
return 第一個, 第二個

# TODO 3：tone_on / tone_off

**必做**　**和 01 的「嗶一聲」一樣**

按網頁琴鍵 → 蜂鳴器發聲，響多久由工具庫計時

```
def tone_on(pitch):
    # 虛擬碼：
    # 音高設成 pitch
    # 音量設成 VOLUME

def tone_off():
    # 音量設成 0

# ⚠ 不要寫 time.sleep
```

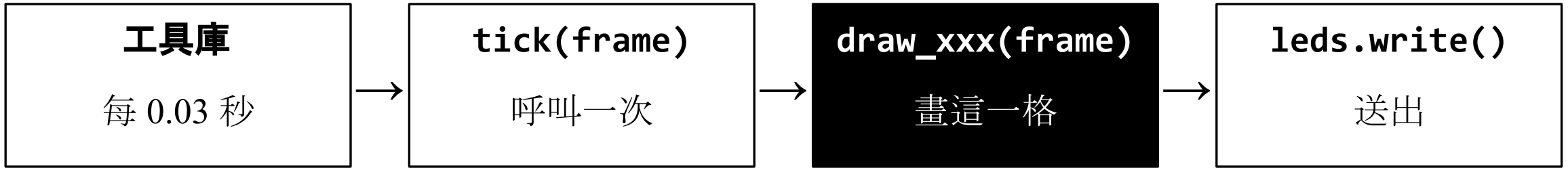
**測試**　按網頁琴鍵；連線、斷線時也會嗶一聲

### 和 01 比一比

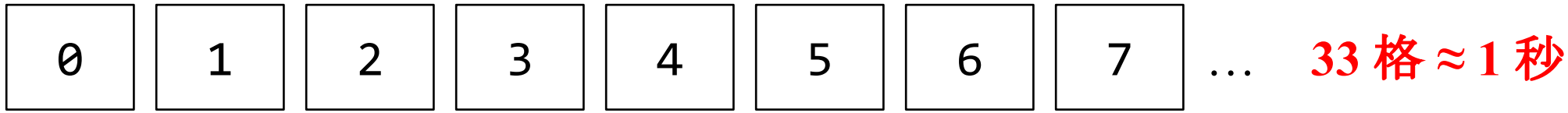
```
buzzer.freq(880)
buzzer.duty_u16(3000)
time.sleep(0.2)
buzzer.duty_u16(0)
```

880 → pitch
3000 → VOLUME
sleep 不寫：工具庫計時，時間到呼叫 tone_off

# frame 是什麼？




**frame = 第幾格，每次 +1：**




**33 格 ≈ 1 秒**

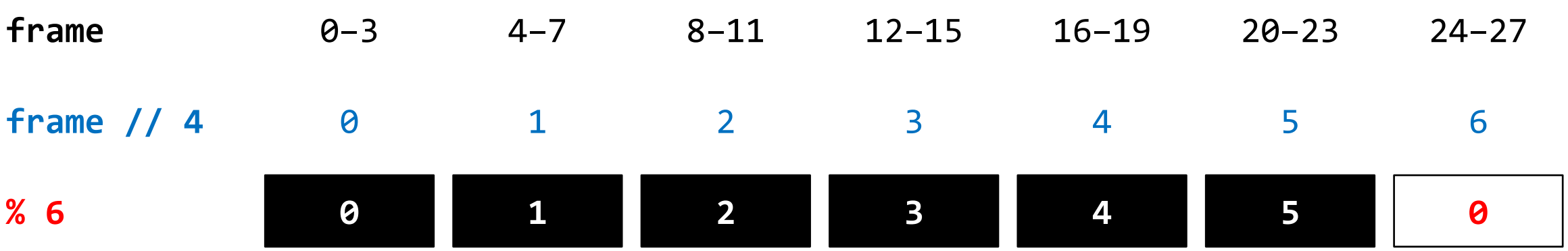
## 就像遊戲技能的冷卻時間（CD）

放完一次技能，要等 CD 跑完才能再放。跑馬燈也一樣：亮一顆之後，要等一段時間才換下一顆。

**02：**time.sleep(0.1) 停下來等　　**儀表板：**不能停，改成數 frame，數到 4 格才換

# 現在輪到第幾顆？

```
position = (frame // 4) % 6
```




**// 4：每 4 格才換一顆**

// 是「除完只留整數」：0~3 都是 0、4~7 都是 1
就是技能 CD：數滿 4 格（約 0.12 秒）才換下一顆

**% 6：只有 6 顆燈，數到 6 就回到 0**

% 是「除以 6 的餘數」：6 → 0、7 → 1
所以 position 永遠在 0 ~ 5 之間繞圈

# TODO 4：draw_chase

**必做** **和 02 的跑馬燈幾乎一樣**

工具庫每 0.03 秒呼叫一次：只要畫出「這一格」

```
def draw_chase(frame):
    # 虛擬碼：
    position = (frame // 4) % 6
    # 對每一顆 number：
    #   如果 number 是 position：
    #     led_colors[position] 的顏色
    #   否則：(0, 0, 0)
```

**⚠ 不要寫 time.sleep / while True**

燈是黑的？先用「全部彩燈」設一個顏色

**測試** 網頁按「跑馬燈」

**和 02 比一比**

```
leds.fill(OFF)
leds[number] = GREEN
leds.write()
time.sleep(0.1)
```

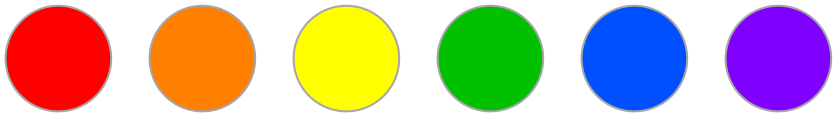
number → position
GREEN → apply_brightness(
led_colors[position])
write、sleep 交給工具庫

# 讀讀看：彩虹是怎麼做的

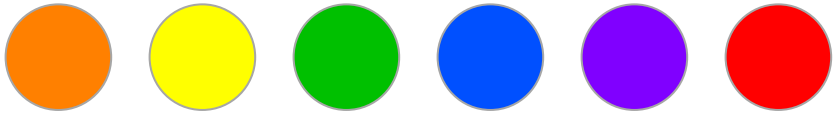
已完成　和跑馬燈一樣是「一格一格畫」

**6 顆燈各用一個顏色，每 4 格整排往前移一格**

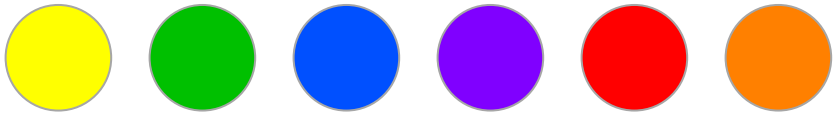
frame 0–3



frame 4–7



frame 8–11



← 每 4 格往前移一格

和跑馬燈一樣用 // 4 數 CD、用 % 6 繞圈

## RAINBOW 是一串顏色

```
RAINBOW = [
    (255, 0, 0),
    (255, 128, 0),
    (255, 255, 0),
    (0, 255, 0),
    (0, 0, 255),
    (128, 0, 255),
]
```

紅、橙、黃、綠、藍、紫
想換顏色？改這 6 組數字

# 讀讀看：彩虹的程式

已完成　公式和跑馬燈幾乎一樣

```
def draw_rainbow(frame):
    shift = frame // 4                                  # 往前移了幾格（CD）
    for number in range(6):
        color = RAINBOW[(number + shift) % 6]   # 第 number 顆用哪個顏色
        leds[number] = apply_brightness(color)
```

**`shift = frame // 4`**

和跑馬燈一樣：每 4 格加 1

**`(number + shift) % 6`**

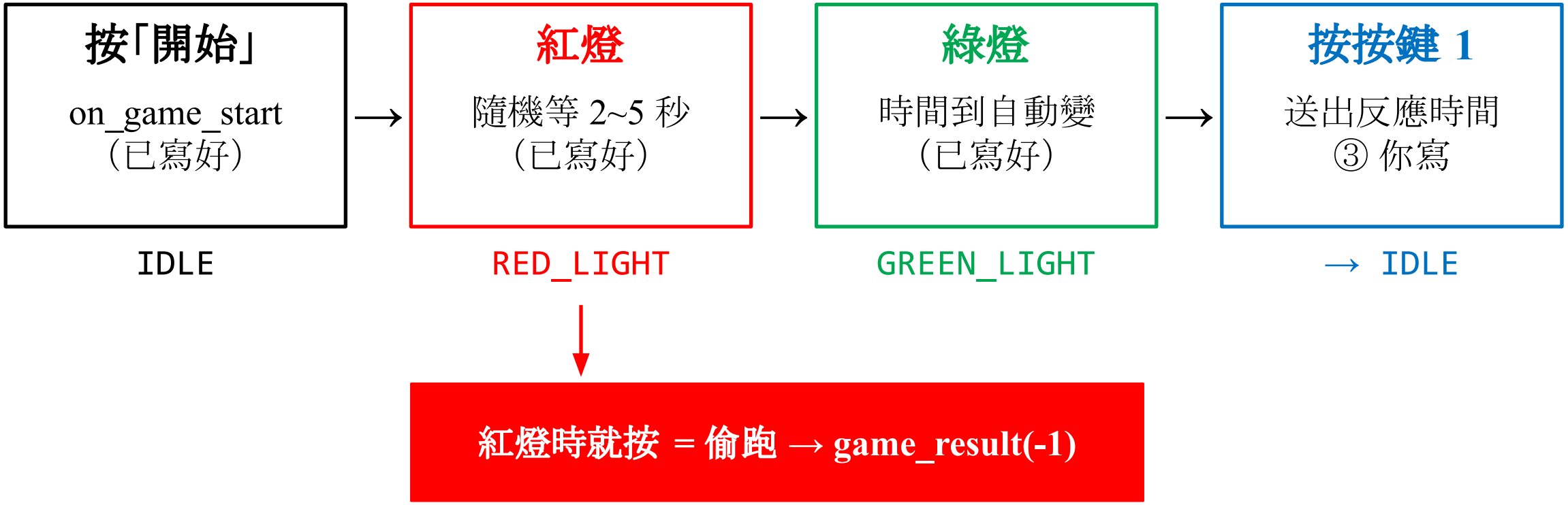
第 number 顆往後找 shift 個顏色
超過 5 就回到 0，所以顏色會一直轉圈

**試試看**

把 // 4 改成 // 2 → 轉得更快；呼吸燈 draw_breathe 也已經寫好，可以讀讀看

# 挑戰：小遊戲反應力（流程）

**挑戰**　和 03 的按鍵 2 一樣，抓「剛剛被按下」




**你要寫的：**② 紅燈／綠燈的顏色　③ 按鍵 1 被按下時，送出結果

# 挑戰:小遊戲反應力(程式)

**挑戰** 寫在 draw_game(frame) 裡

```
# ② 如果 RED_LIGHT:6 顆紅;否則:6 顆綠

# ③ button1_now = 讀 button1 的值
#   如果 上一次 0、這一次 1:
#     如果 RED_LIGHT:game_result(-1)
#     否則:game_result(反應時間)
#     game_state = IDLE
#   button1_before = button1_now
# 反應時間 = time.ticks_diff(now, green_at_ms)
```

**測試** 網頁「小遊戲:反應力」按開始

加分:成功 mango_ble.beep(1320, 120)

### 和 03 比一比

```
if (before == 0
    and now == 1):
    ...
before = now
```

最後記得更新 button1_before

ticks_diff(現在, 綠燈時刻)
= 綠燈後過了幾毫秒

# 網頁和程式的結構

| | |
|---|---|
| morning/project/ | |
| board/ | **放到板子上執行** |
| dashboard.py | **你的函式: TODO 都在這裡** |
| mango_ble.py | 板子端藍牙工具庫(Upload) |
| web/ | **電腦開 http.server, 瀏覽器打開** |
| index.html | 網頁上有哪些卡片、按鈕 |
| app.js | 按下去做什麼:mango.write(...) |
| mango-ble.js | 網頁端藍牙:和 mango_ble.py 對話 |
| style.css | 顏色、排版 |

**做 TODO 只要改 dashboard.py;自由發揮前, 先讀每個檔案開頭的 說明**

# 自由發揮:和 AI 一起做

**按鍵鋼琴**

按住 Key1 / Key2
各彈一個音

**手電筒**

按住 Key2
6 顆燈全白

**改網頁**

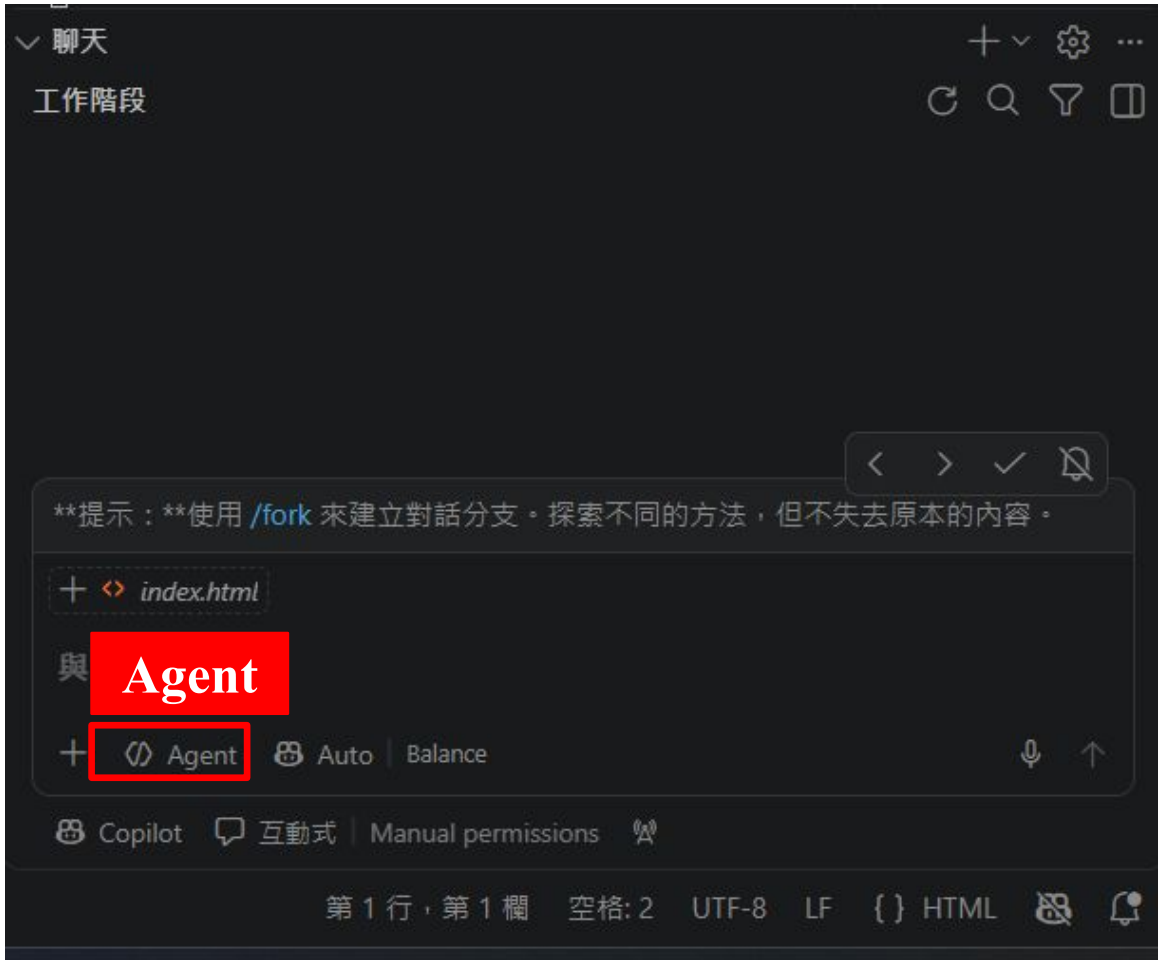
換顏色、改排版

**新功能**

網頁多一個按鈕



# VS Code 內建的 AI Agent




**1 打開聊天**

Ctrl+Alt+I, 或右上角的聊天圖示

**2 模式選 Agent**

Agent 會自己讀檔案、直接改程式

**3 描述你想要的效果**

例如：網頁多一個按鈕, 按下去 6 顆燈變白色

**改完記得 Stop → Run → 網頁重新連線**

## 網頁儀表板實作

# 30 分鐘

TODO 1~4 必做, 做完挑戰小遊戲, 或和 AI 一起自由發揮

11:45

# 展示時間

2 ~ 3 位同學分享小遊戲或自由發揮的作品

# 今天學到的



**輸出**

彩燈 write()
蜂鳴器 freq()



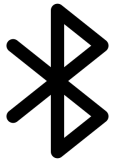
**輸入**

按鈕 value()
抓按下的瞬間



**腳位名稱**

Pin("KEY1")
= Pin(30)



**網頁遙控**

網頁按一下
→ 你的函式被呼叫

午休

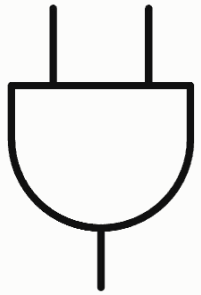
# 下午:舵機實作

MANGO BLOCK × Hiwonder 匯流排舵機

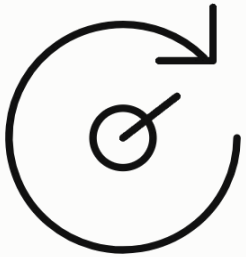
# 舵機實作

## MANGO BLOCK × Hiwonder 匯流排舵機

# 目標



接好一顆



連上它
讓它轉



三顆一起轉

# 準備材料




Bus Servo x 3

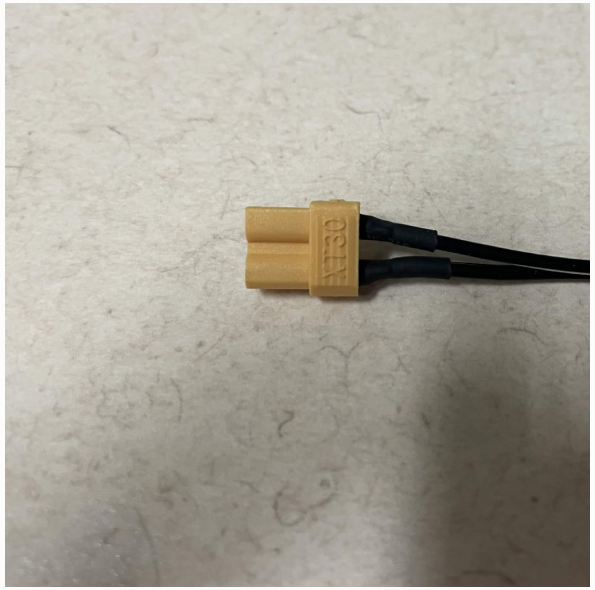


舵機串接線 x 3

# 準備材料

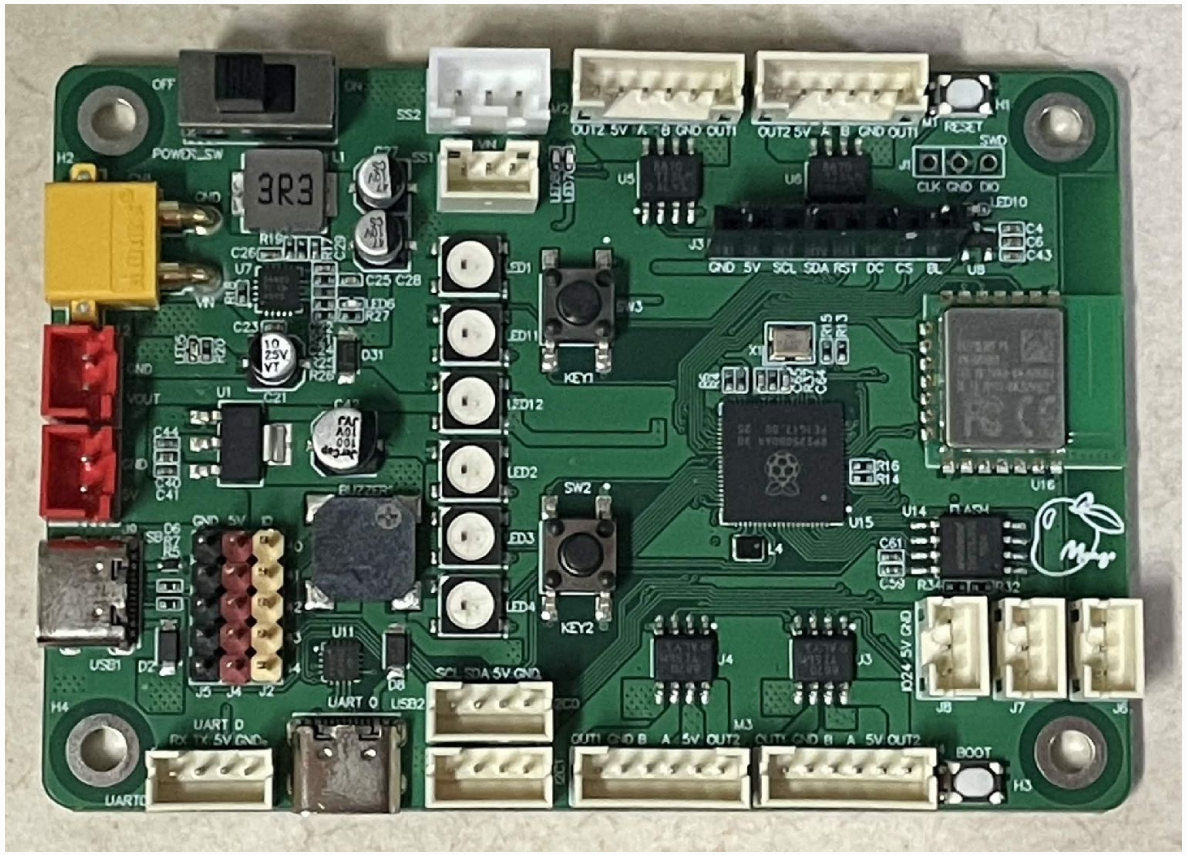


Type-C USB x1




XT30 供電線 x1

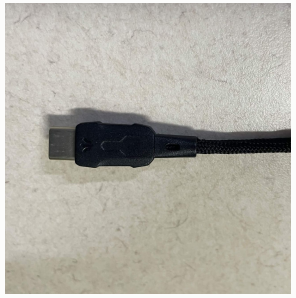
# 準備材料



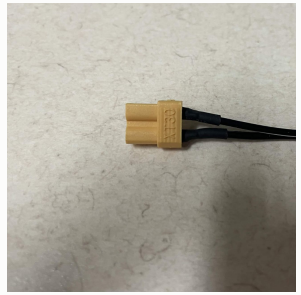

**MANGO BLOCK x1**

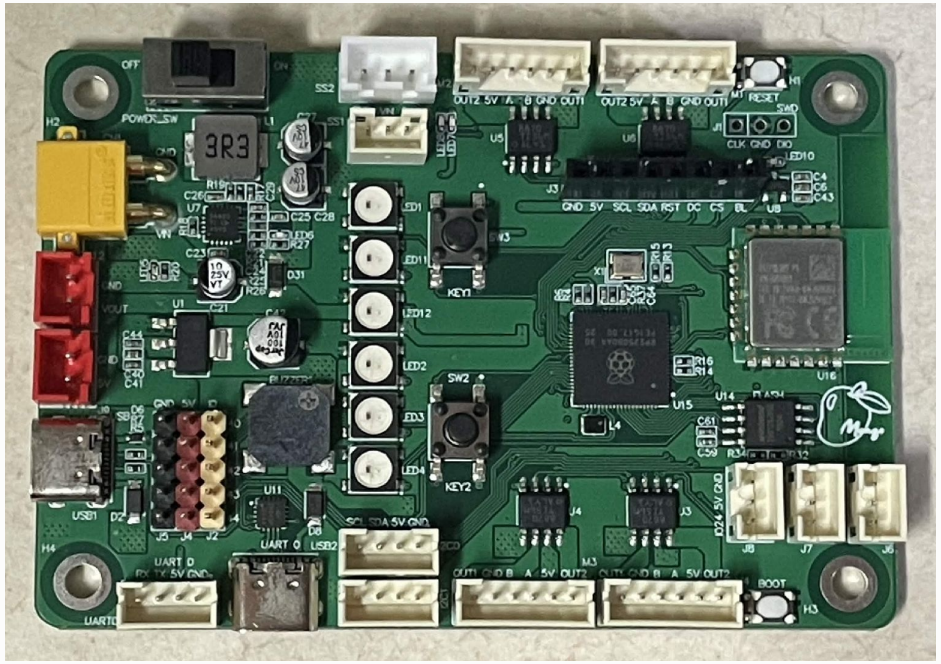
# 檢查清單









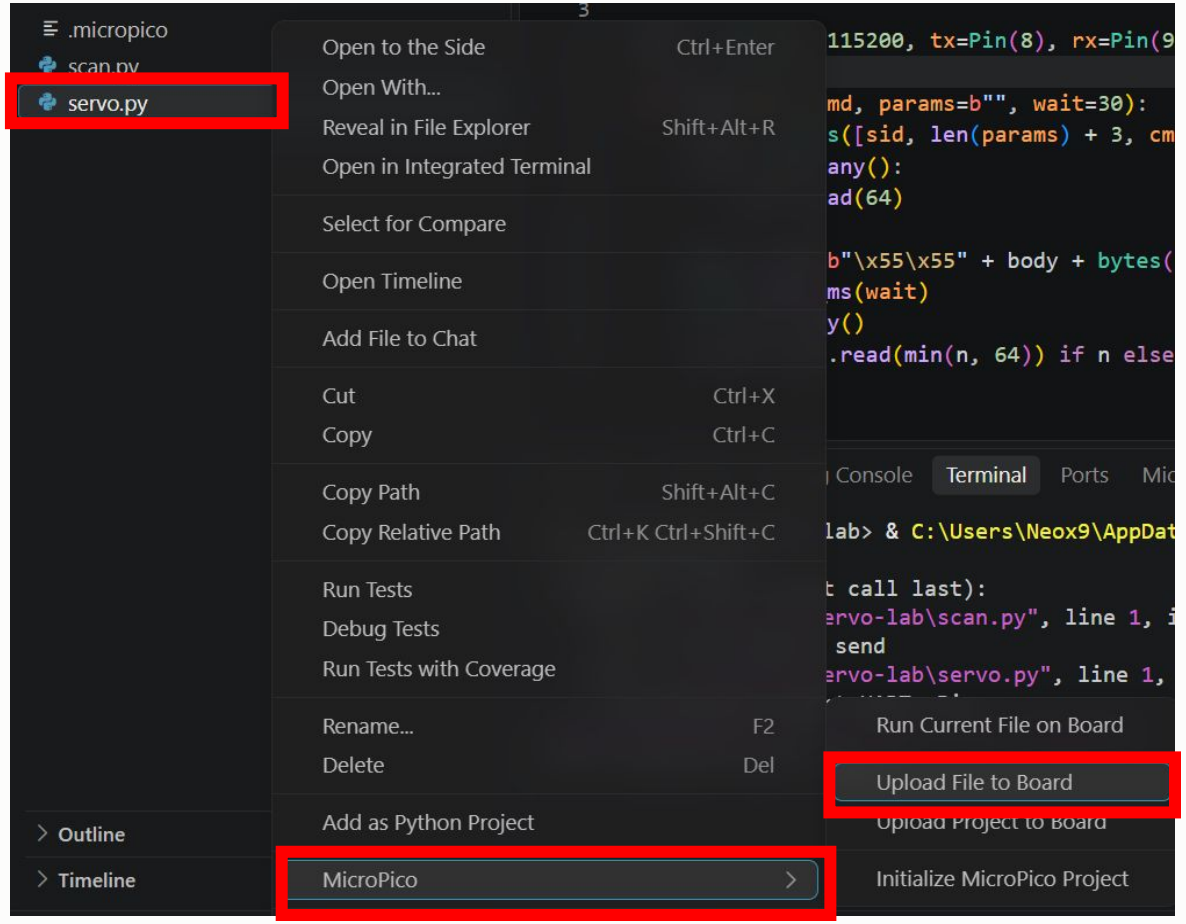
# 記得斷電

做任何電路上的動作時

請把電源關掉或拔掉電源線

# 透過 Vscode 跑程式






1

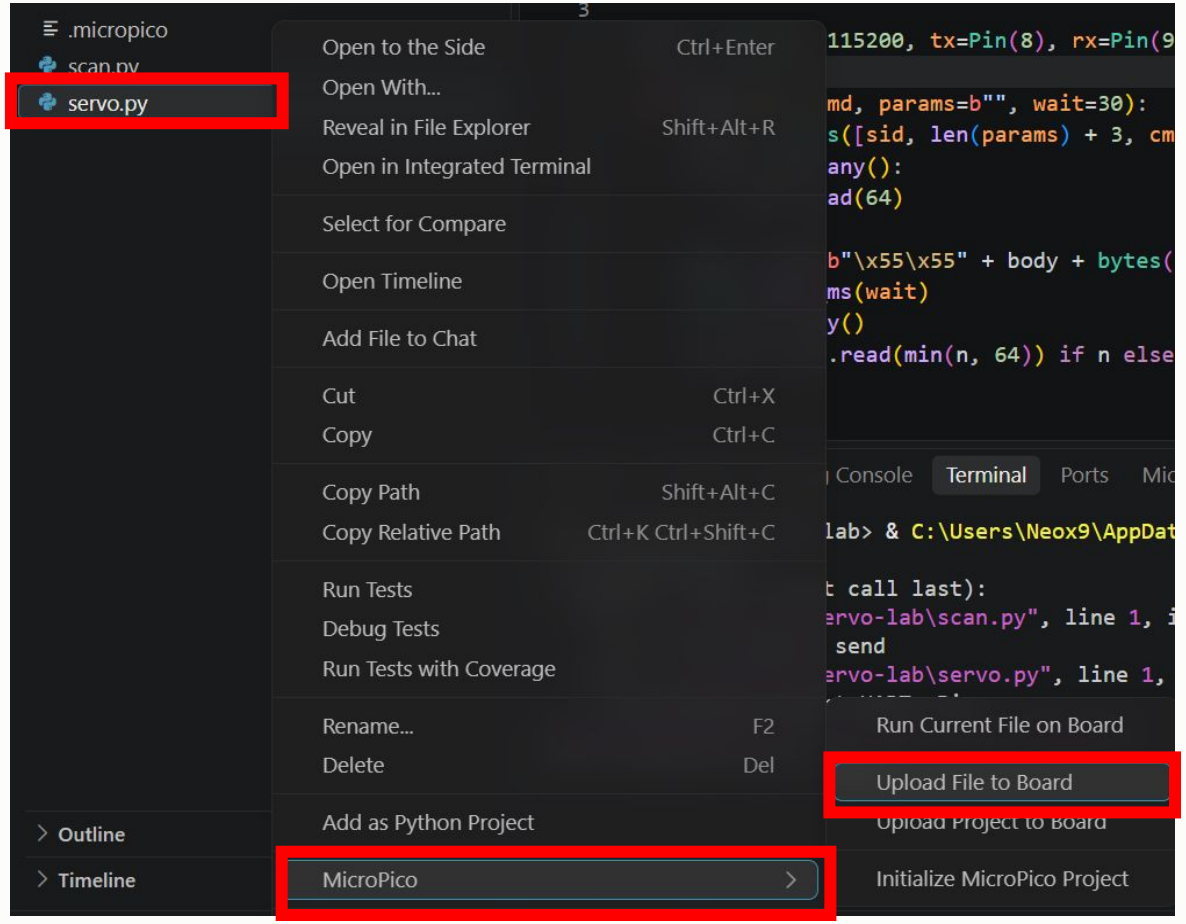
## 上傳 servo.py

在檔案上按右鍵

→ Upload file to Board

# 透過 Vscode 跑程式






2

## 讓蜂鳴器叫

在 vREPL 輸入：

```
from servo import beep
beep(1)
```

Part 1

# 單顆舵機

接線 · 讀取 · 讓它轉

# 認識舵機






Hiwonder

## HTD-45H

**45 kg・cm**

**高壓匯流排舵機**

# 一般舵機 (PWM) vs 匯流排舵機 (Bus Servo)



## 一般 PWM 舵機

一顆佔一支腳

只能下命令

讀不到現在的角度

## 匯流排舵機

一條線串很多顆

用封包 + ID 指定誰

讀得到位置、電壓、溫度

# 一般舵機 (PWM) vs 匯流排舵機 (Bus Servo)








# 規格



45 kg・cm
扭力

9–12.6 V
工作電壓

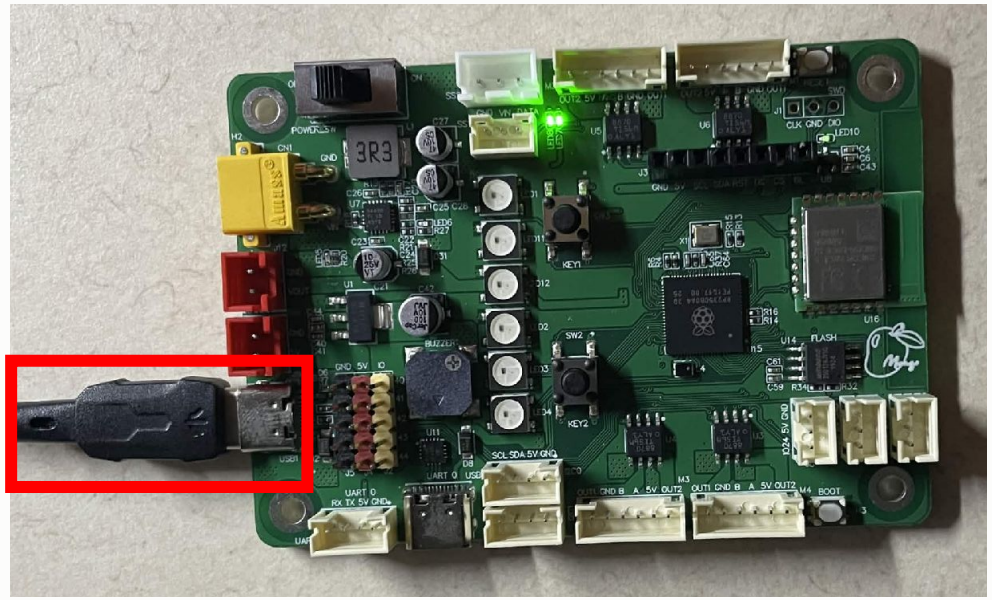
0–240°
位置 0–1000

ID 0–253
出廠是 1

115200
baud ・ 單線半雙工

# 接線





1

## 插上 Type-C

板子開機，VS Code 連上

# 接線



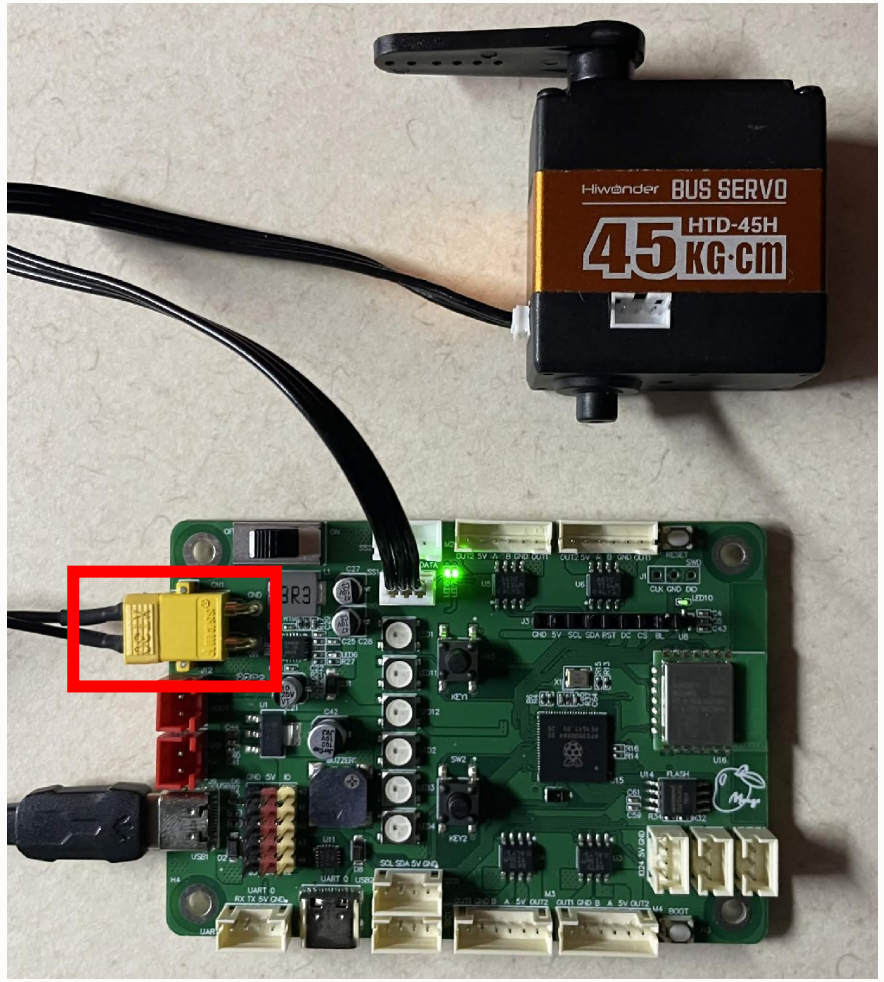



2

## 舵機連接 SS1

# 接線






3

## XT30 接 12V 電源

**開關先保持 OFF**

# 接線



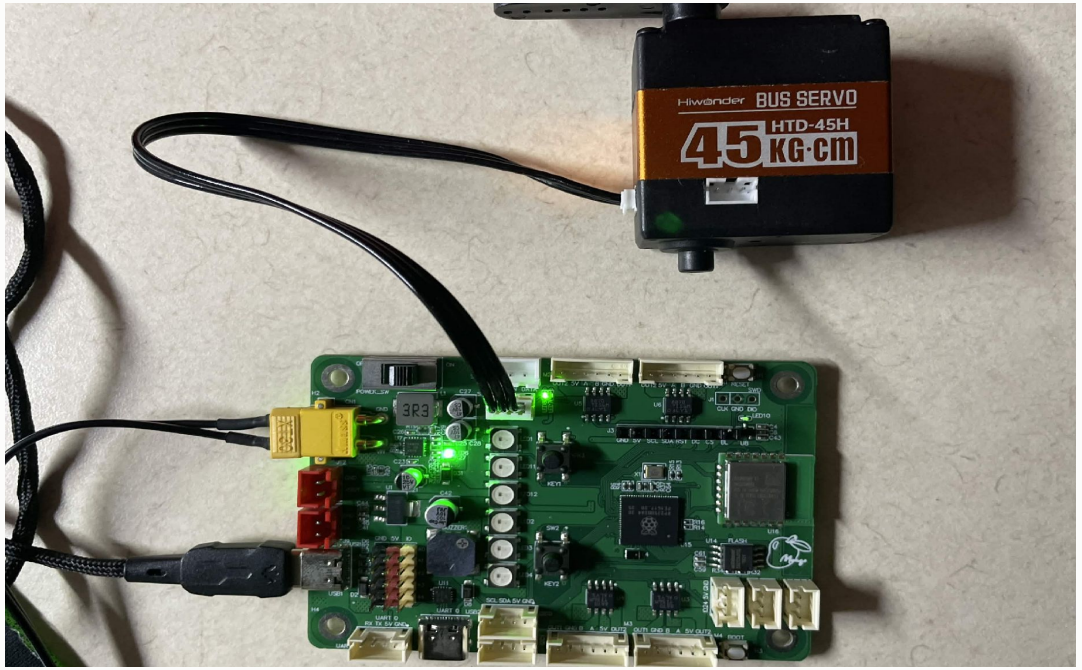



4

## POWER_SW 撥到 ON

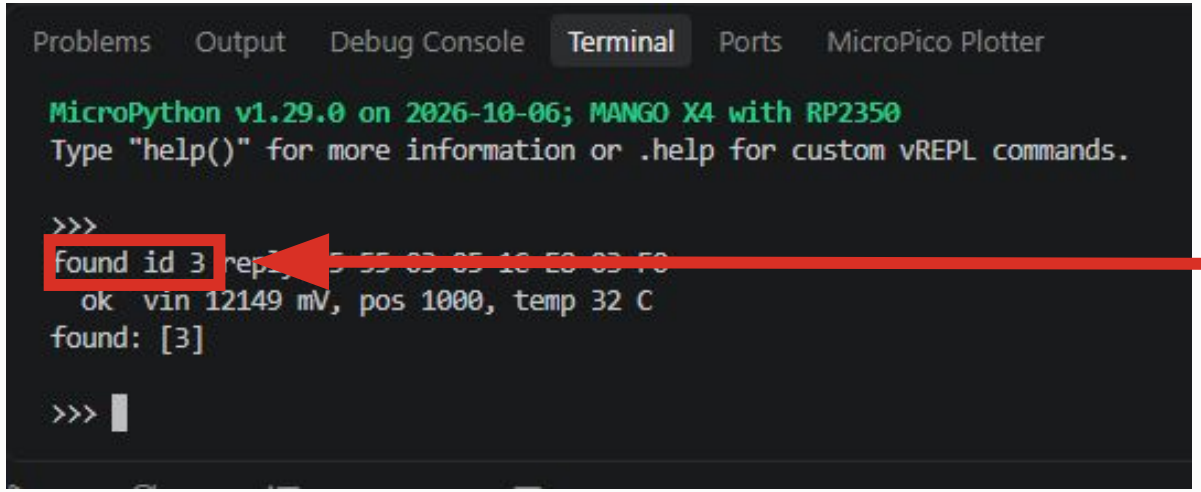
# 接線總覽


電腦
Type-C
MANGO X4
SS1
舵機
12V 電源
XT30


Hiwonder BUS SERVO
HTD-45H
45 KG·cm

# 先讀，不要轉

在 VS Code 跑 scan.py ・ 掃描 ID 0–9，得到馬達 ID



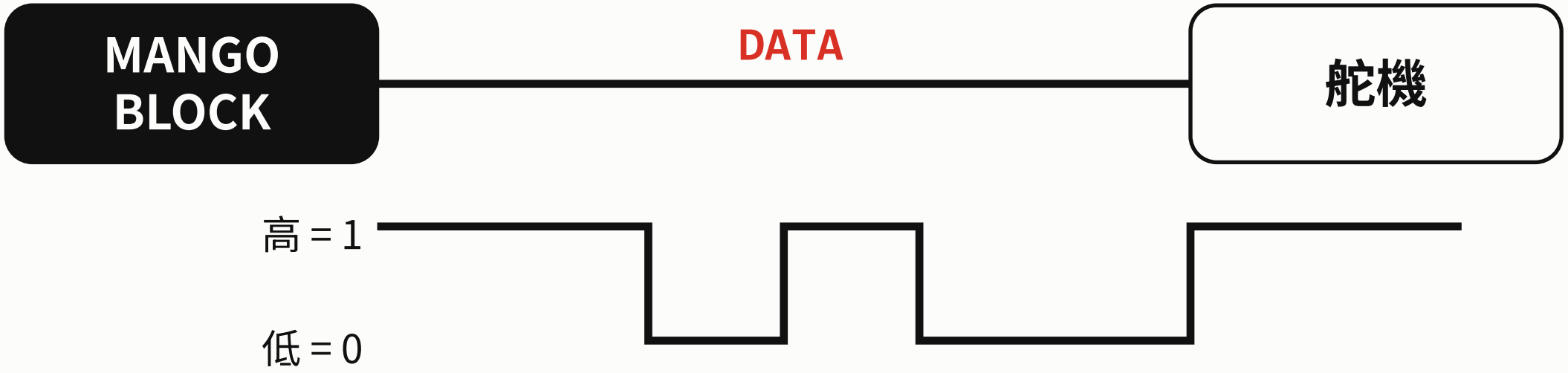

**記住這個 ID**

- **掃得到 ID**
- **電壓 12v 左右**
- **溫度 30°C 上下**

# 通訊

# DATA 線怎麼傳訊息





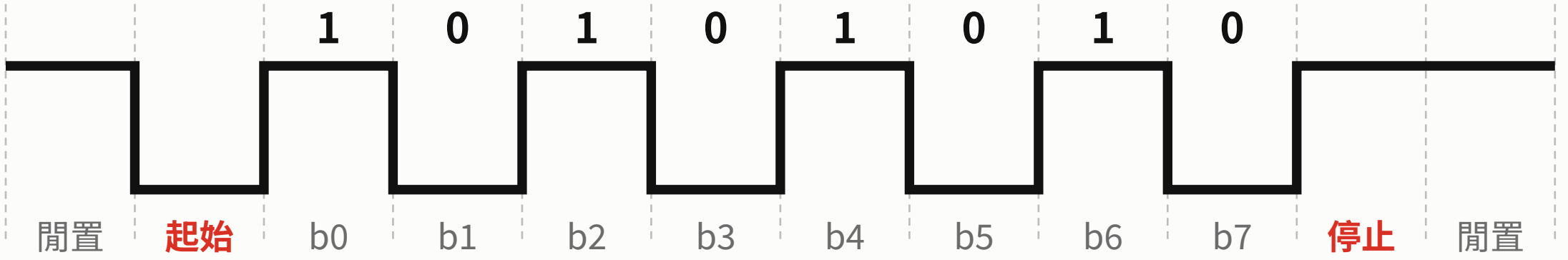

**真正傳資料的只有一條 DATA 線**

線上只有兩種狀態：高電位（1）、低電位（0）

沒在傳的時候，線保持**高電位**

# DATA 線怎麼傳訊息






送出 0x55 = 0101 0101（低位先送：1, 0, 1, 0, 1, 0, 1, 0）

**起始位元：先拉低一格，告訴對方「要開始了」**

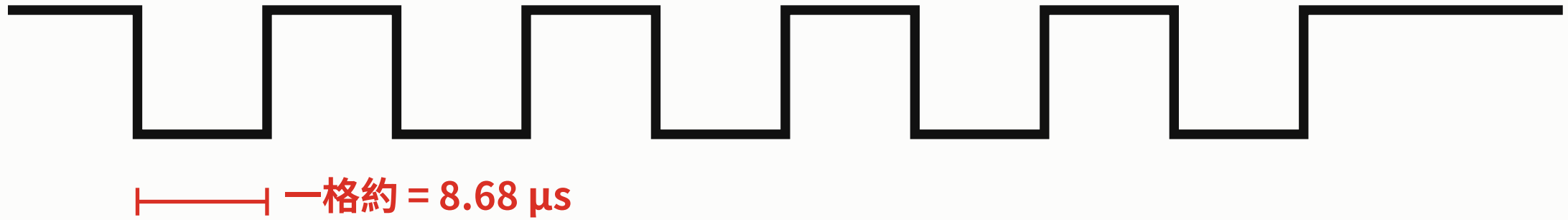
**資料位元：8 格，就是一個位元組，最低位先送**

**停止位元：回到高電位一格，代表「這個位元組結束」**

**一個位元組 = 10 格**

# DATA 線怎麼傳訊息






板子 115200 ↔ 舵機 115200
讀得懂

板子 115200 ↔ 舵機 9600
讀錯格，變亂碼

**115200 baud**：每秒送 115200 格，一格約 8.68 µs

**兩邊的速度一定要一樣**

# DATA 線怎麼傳訊息



| 55 | 55 | 01 | 07 | 01 | F4 | 01 | E8 | 03 | 16 |
|---|---|---|---|---|---|---|---|---|---|

**封包 = 一串位元組**

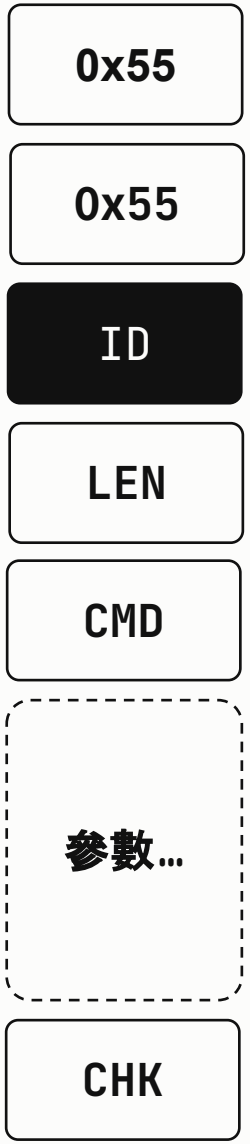
每個位元組都用「起始 + 8 位元 + 停止」

送 move(1, 500, 1000) 的封包有 10 個位元組 = 100 格

115200 baud 下大約 0.87 ms 就送完

# 封包

# 一個封包長這樣




**開頭**（兩個 0x55）

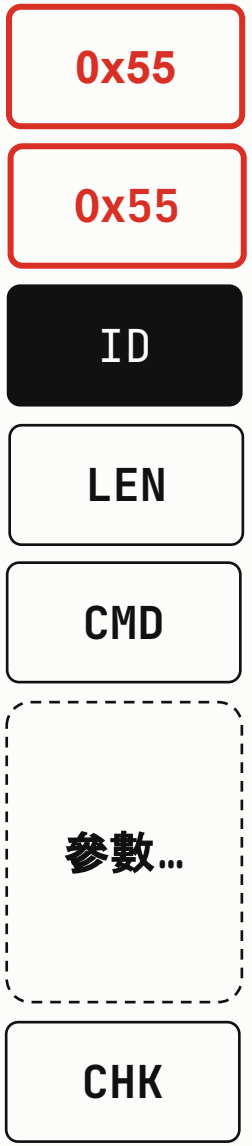
**Servo ID** - 要發送指令的馬達 ID

**LENGTH** = ID + LEN + CMD + PARAMETERS 多少 Bytes

**COMMAND** - 給舵機的指令

**PARAMETERS** - 參數 (看指令種類，有的可以留空)

**CHK = ~(ID + LEN + CMD + 參數) & 0xFF**



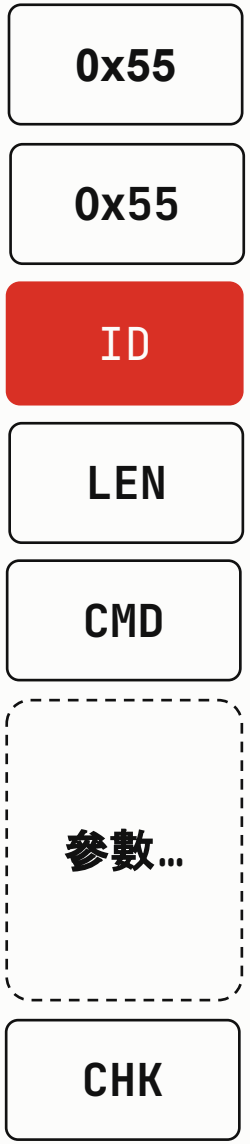

# 開頭

固定兩個 0x55，代表「封包從這裡開始」

0x55 = 0101 0101

0 和 1 交錯，接收端容易對準時間

連續兩個 55 很少見，不容易認錯



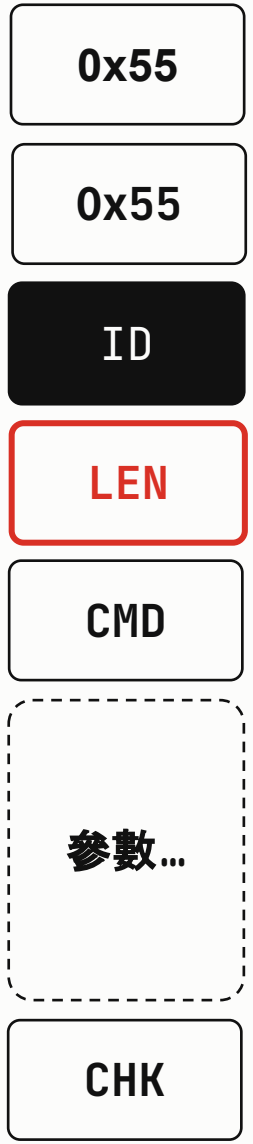

# SERVO ID

**ID 0–253**

**決定要哪一顆舵機執行指令**

**所有舵機都聽得到，只有 ID 對上的那顆會動、會回答**

**254（0xFE）是廣播**




# LENGTH

**LEN = 參數個數 + 3**（ID、LEN、CMD）

**接收端靠它知道後面還要讀幾個位元組**

**讀電壓**　0 個參數 → **LEN 3**

**改 ID**　1 個參數 → **LEN 4**

**移動**　4 個參數 → **LEN 7**

0x55

0x55

ID

LEN

CMD

參數…

CHK

# COMMAND

**CMD 告訴舵機「要做什麼」**

**讀取：26 溫度、27 電壓、28 位置 → 舵機會回覆**

**寫入：1 移動、13 改 ID → 舵機不回覆**

**回覆裡的 CMD 跟送出的一樣，可以用來對答案**



**電壓 • CMD 27**

回傳電壓 mV



**溫度 • CMD 26**

回傳溫度 °C



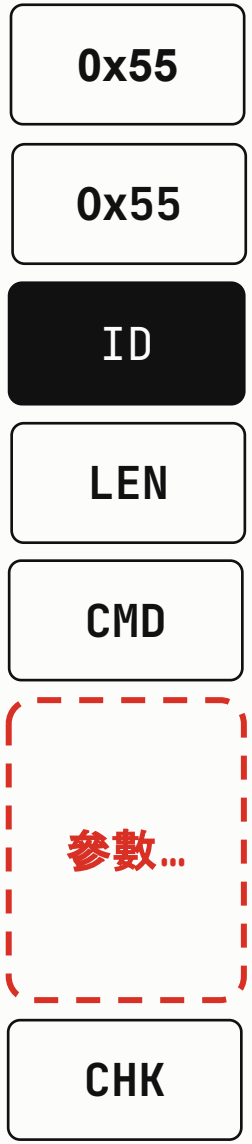
**位置 • CMD 28**

回傳位置0–1000



**角度限制 • CMD 21**

回傳下上限



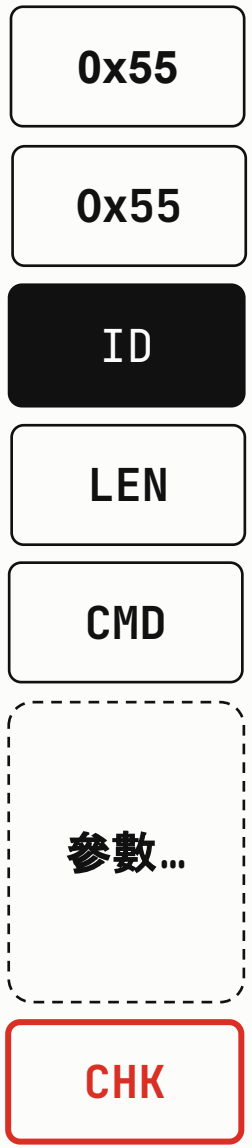

# 參數 PARAMETERS

每個指令要的參數不同，讀取類通常沒有參數

超過 255 的數字拆成兩個位元組，低位元組在前

500 = 0x01F4 → F4 01

1000 = 0x03E8 → E8 03




# CHECK

CHK = ~(ID + LEN + CMD + 參數) & 0xFF

1. 相加：01+07+01+F4+01+E8+03 = 0x1E9
2. 只留最低一個位元組：0xE9
3. 每個位元反過來：~0xE9 = 0x16

收到的一方再算一次，對不上就 PASS

# 把封包送出去

servo.py

```
from machine import UART, Pin, PWM
import time

uart = UART(1, 115200, tx=Pin(8), rx=Pin(9))

def send(sid, cmd, params=b"", wait=30) -> bytes:
    body = bytes([sid, len(params) + 3, cmd]) + params

    while uart.any():
        uart.read(64)

    uart.write(b"\x55\x55" + body + bytes([~sum(body) & 0xFF]))

    time.sleep_ms(wait)
    n = uart.any()
    return (uart.read(min(n, 64)) or b"") if n else b""
```

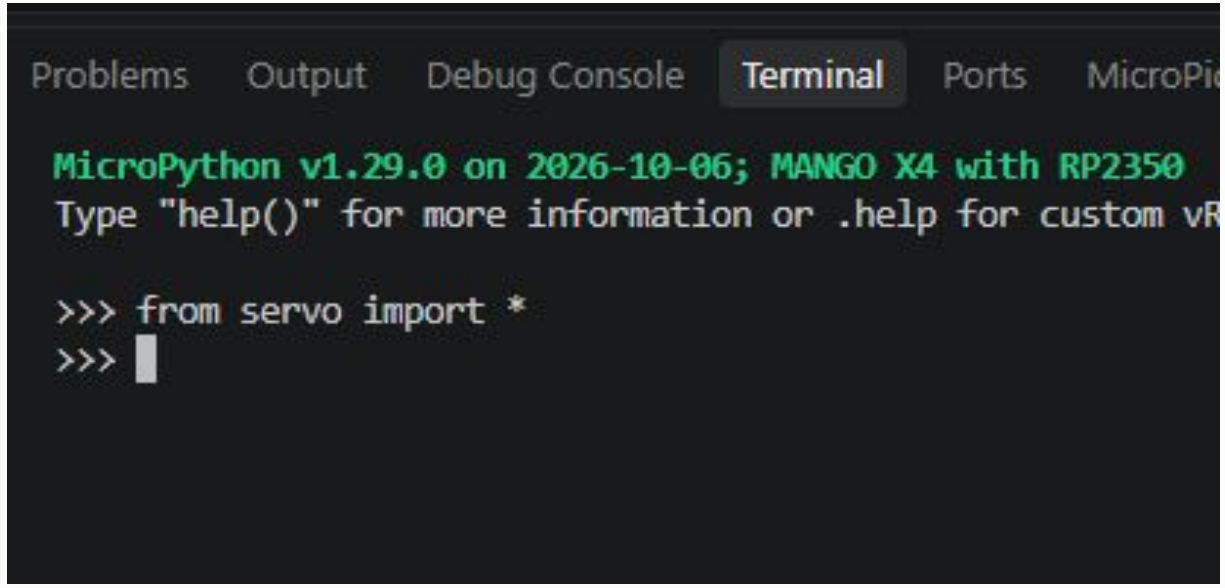
# 讓它轉

Upload servo.py 之後，可以使用這四個指令

| | |
|---|---|
| `move(id, 位置, 毫秒)` | 轉到指定位置，0–1000 |
| `pos(id)` | 讀現在的位置 |
| `vin(id)` | 讀電壓，單位 mV |
| `beep(n)` | 蜂鳴器響 n 聲 |

# 讓它轉






1

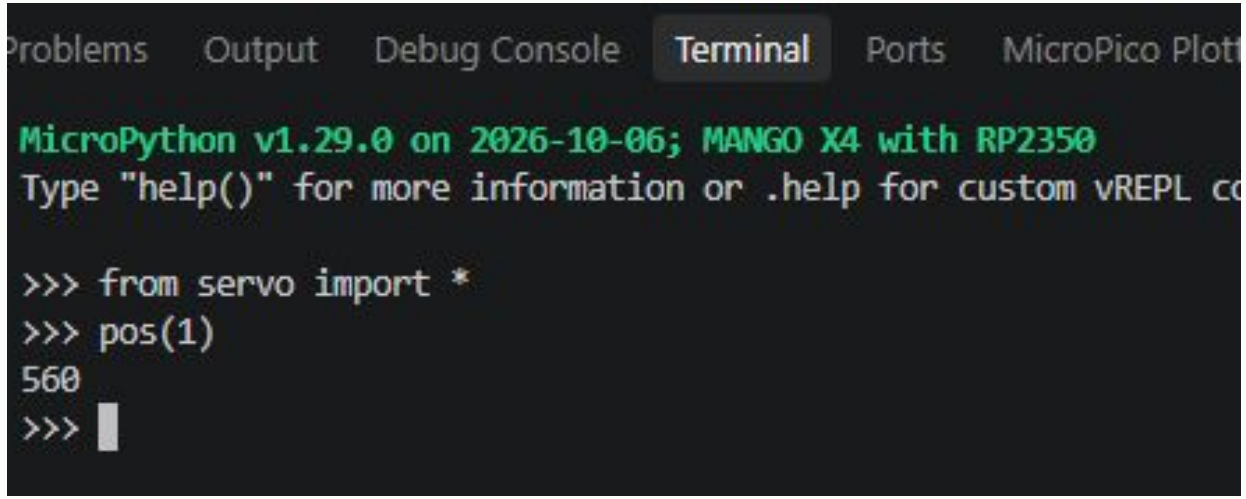
## 匯入指令

```
>>> from servo import *
```

每次重開板子都要打一次

# 讓它轉






2

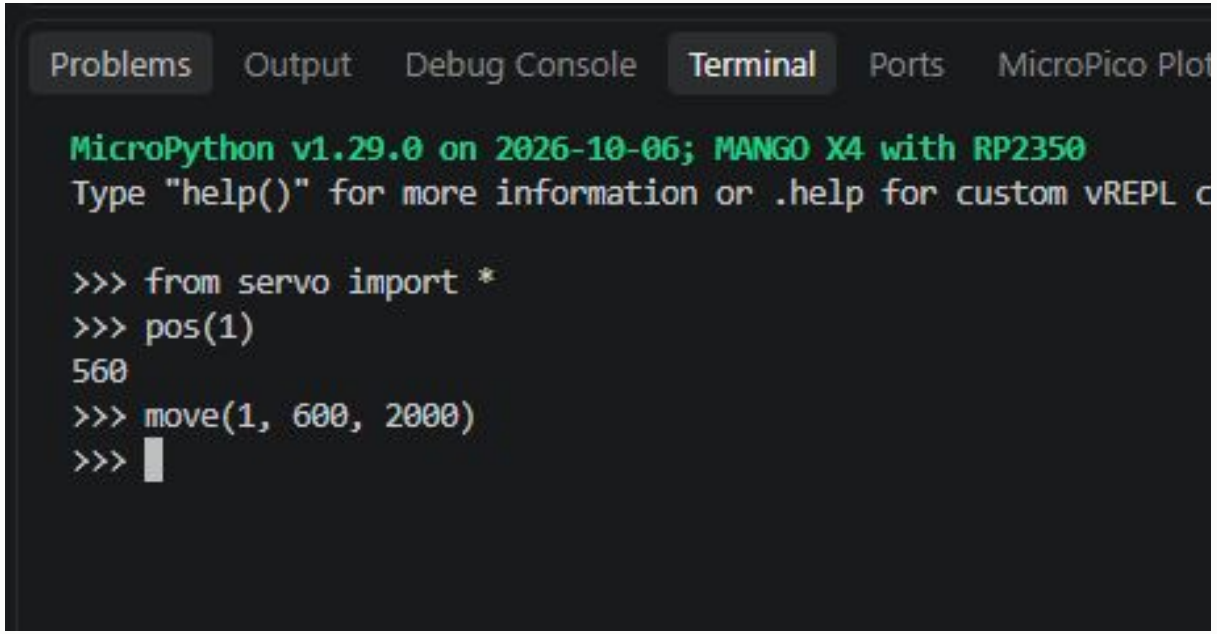
## 讀現在的位置

```
>>> pos(1)
560
```

先知道它停在哪裡

# 讓它轉






3

## 轉到 600

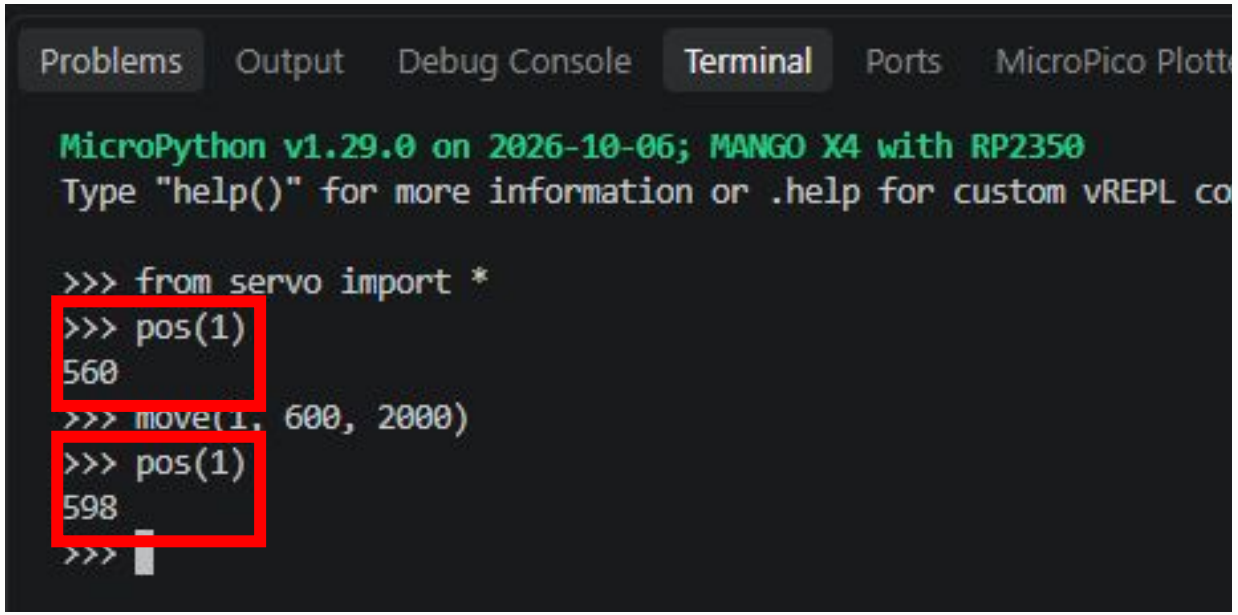
```
>>> move(1, 600, 2000)
```

2 秒轉到 600

手先固定好舵機

# 讓它轉






4

## 確認到了沒

```
>>> pos(1)
598
```

差一點點是正常的

Part 2

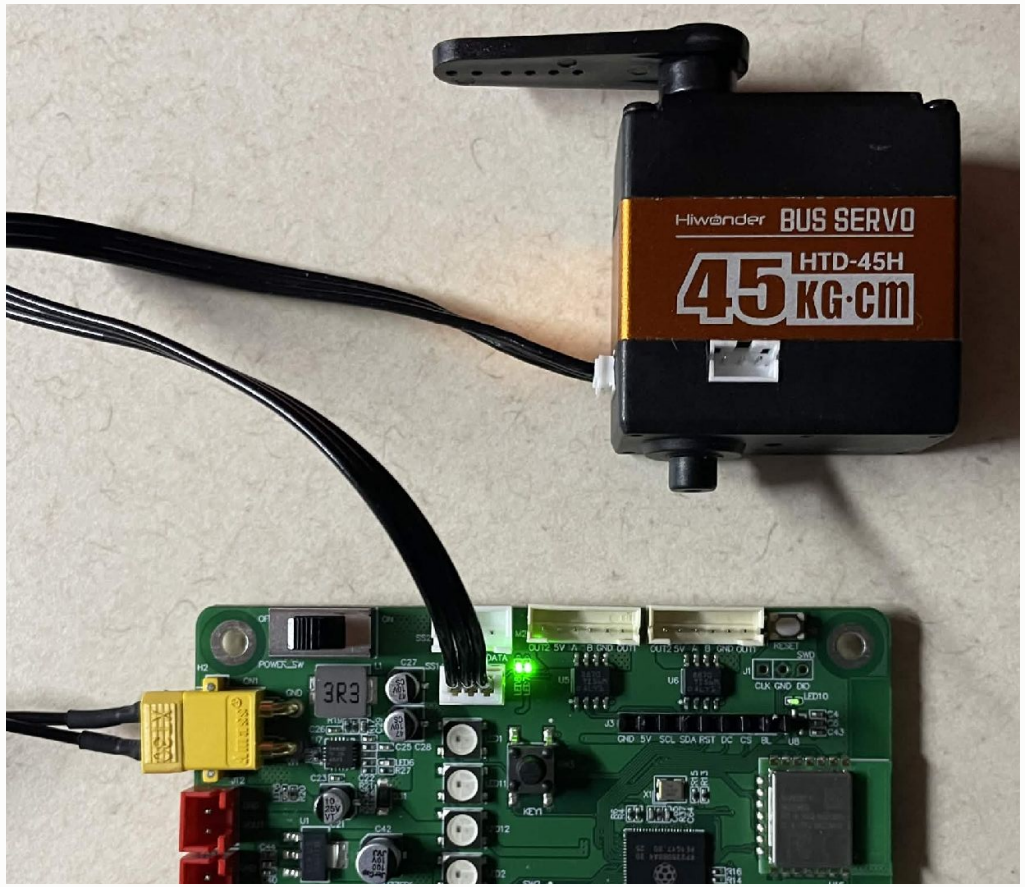
# 多舵機操控

串接 · ID · 輪流轉

# 改 ID

# 改 Servo ID





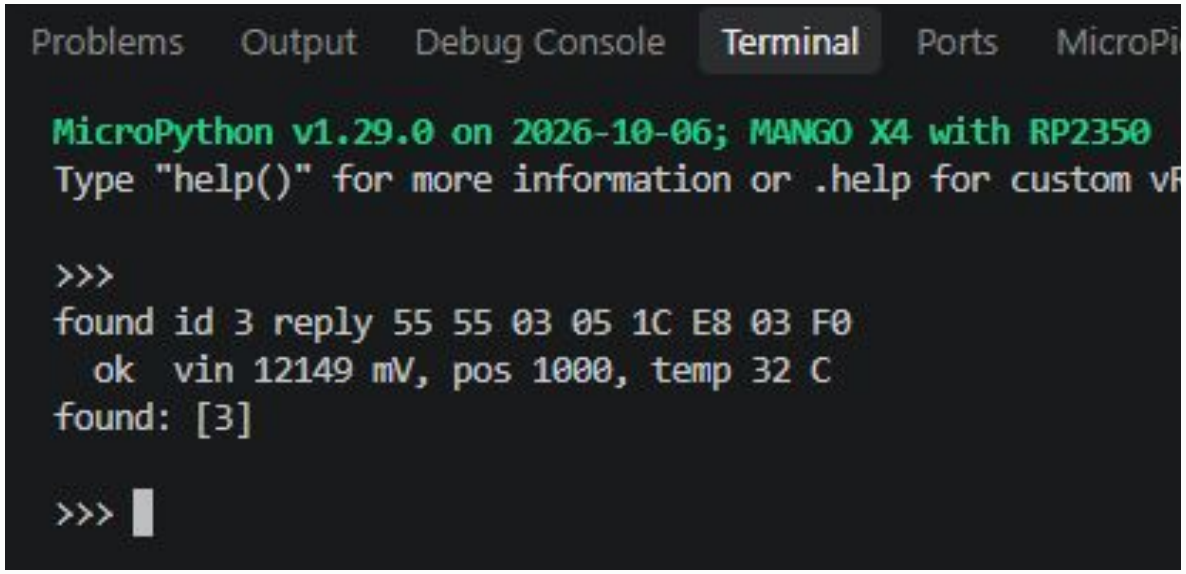

1

## 關電，只留一顆

重複的 ID 只能剩一顆在線上

# 改 Servo ID





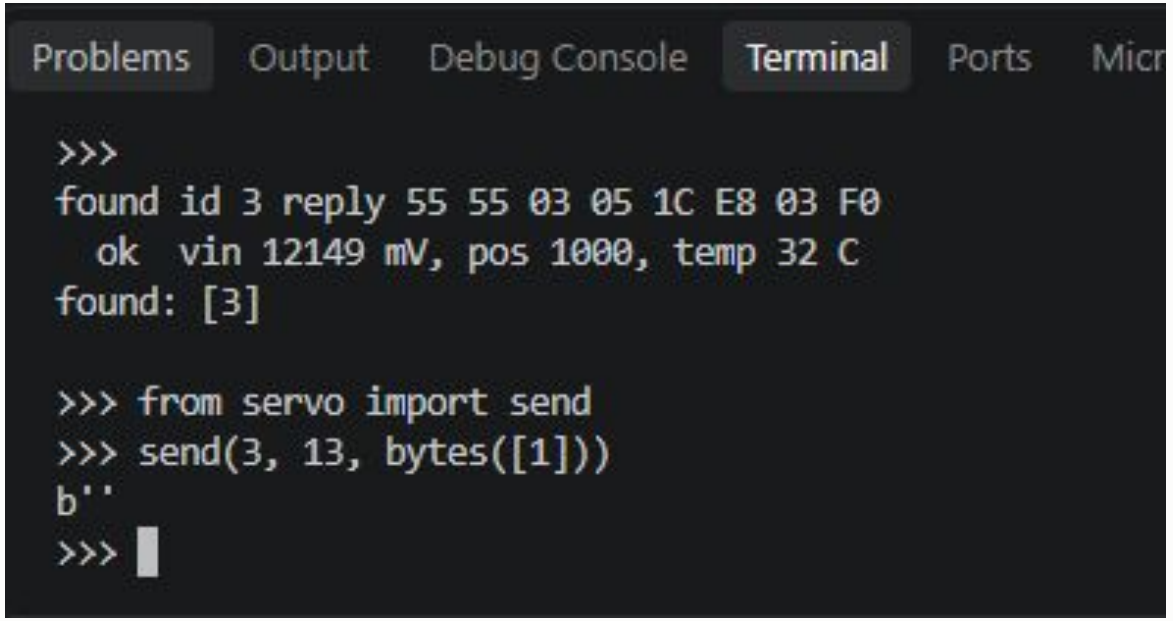

2

## 開電，讀讀看

執行 scan.py

# 改 Servo ID






3

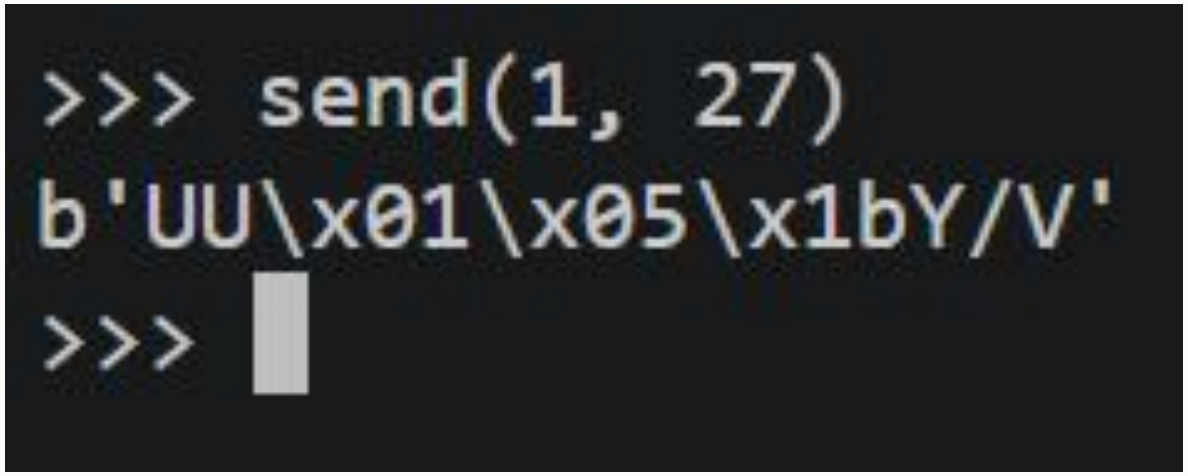
## 送出指令改 ID

```
from servo import send
send(3, 13,
bytes([1]))
```

把 ID 3 改成 1

# 改 Servo ID






4

## 用新 ID 讀

```
send(1, 27)
# 或是跑 scan.py
```

有回應就改好了

## 目標

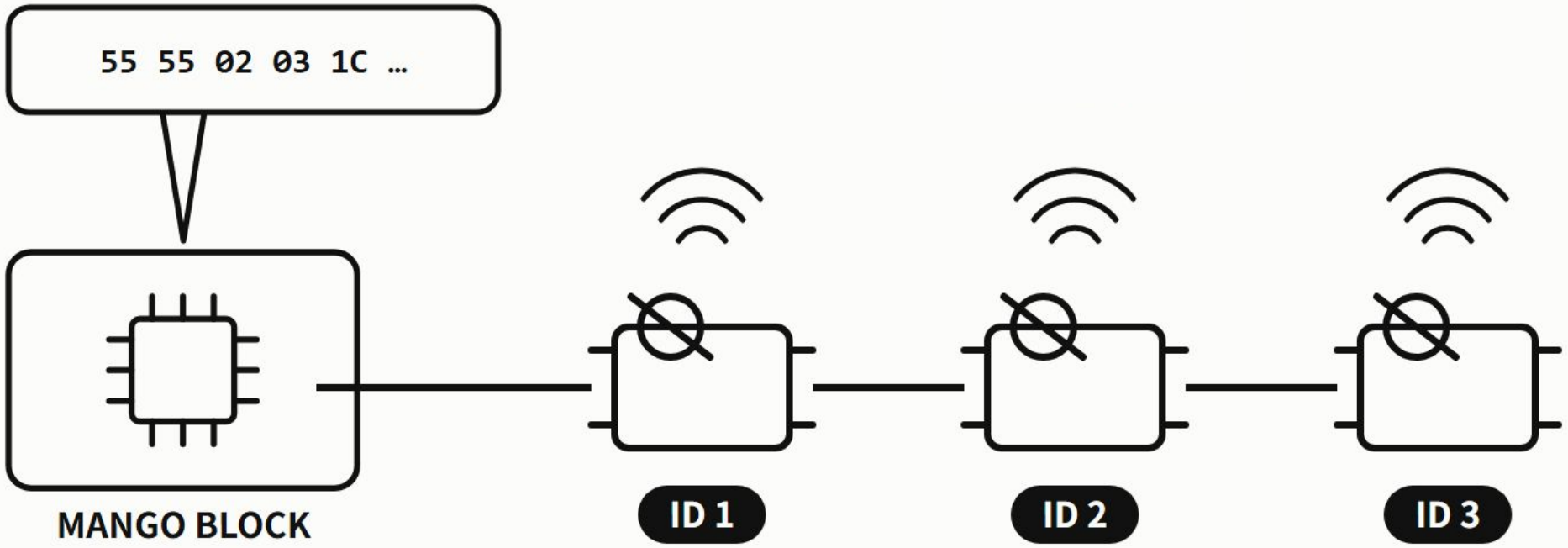
# 將三個舵機 ID
# 改為 1, 2, 3

# 接上他們


Hiwonder BUS SERVO
HTD-45H
45 KG·cm
Hiwonder
45 KG·cm
Hiwonder BUS SERVO
45 KG·cm

# 多舵機通訊

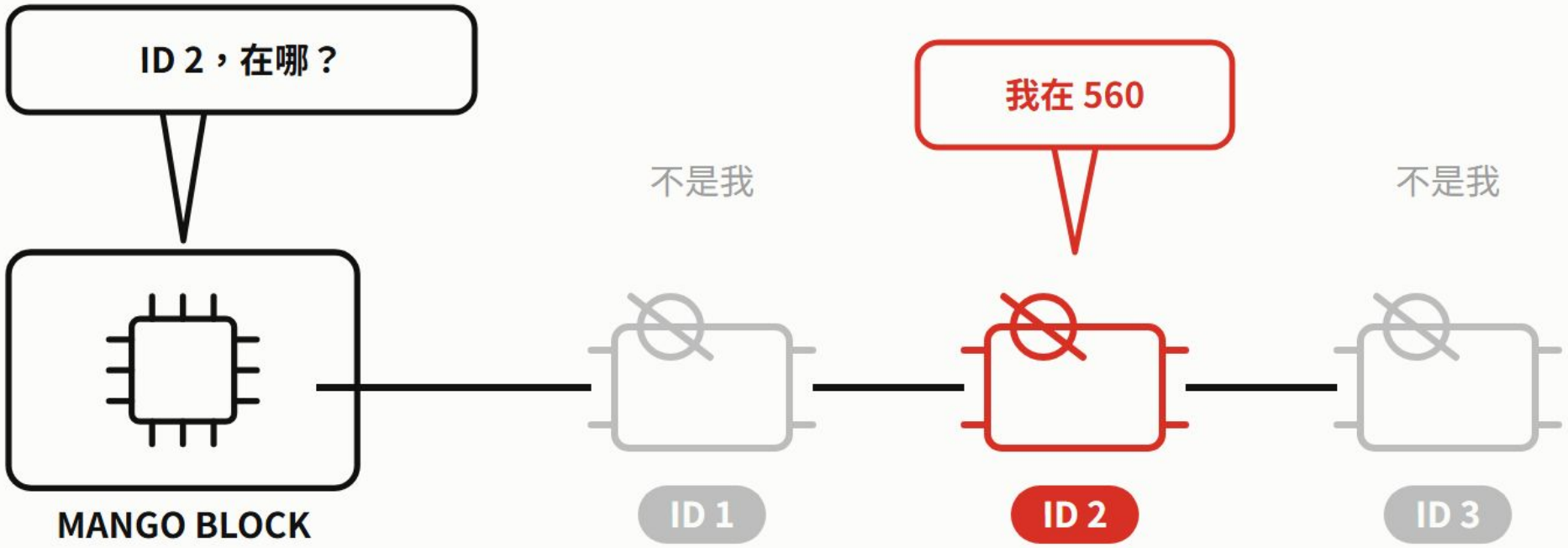





**三顆串在同一條線上，每個封包大家都收得到**

# 多舵機通訊

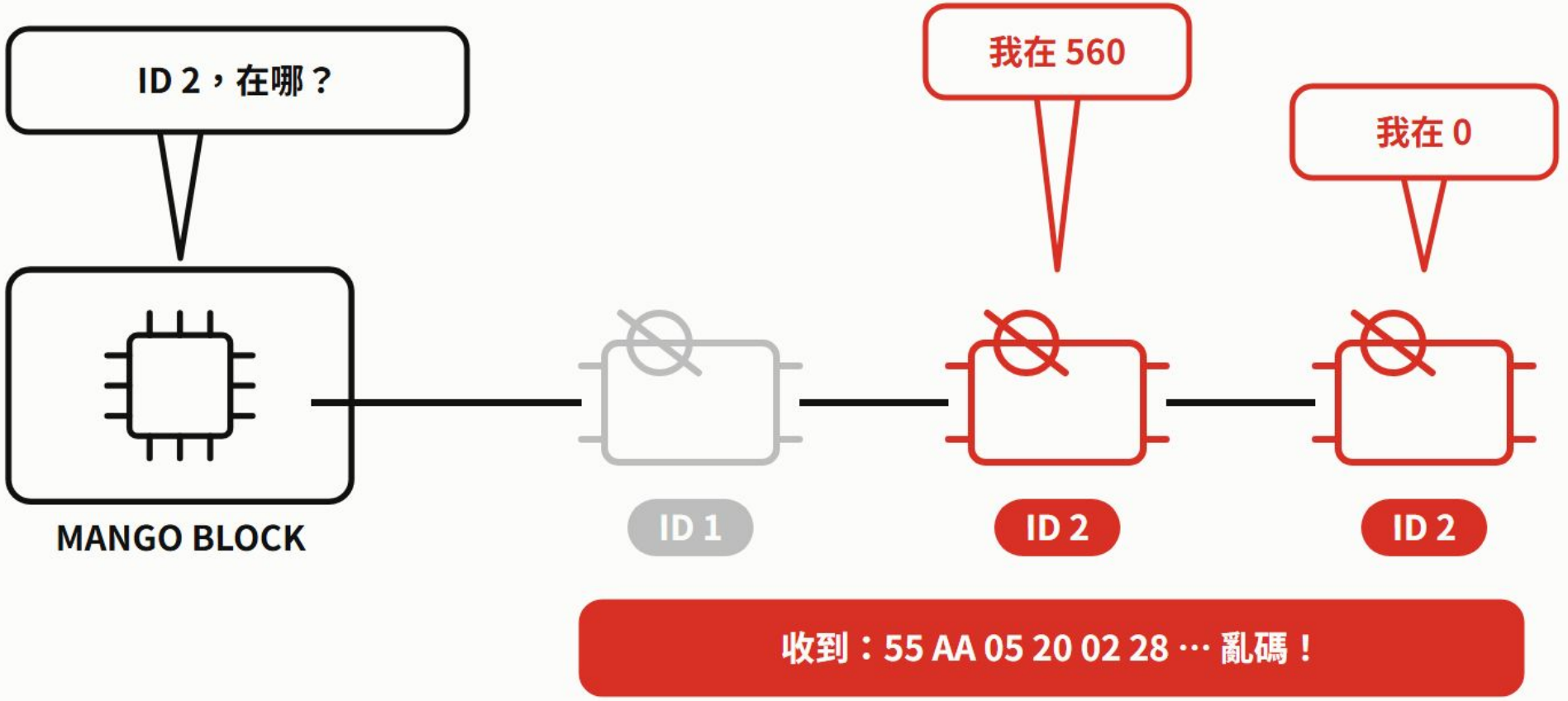





**封包裡的 ID 決定誰回答，其他顆不理它**

# 多舵機通訊

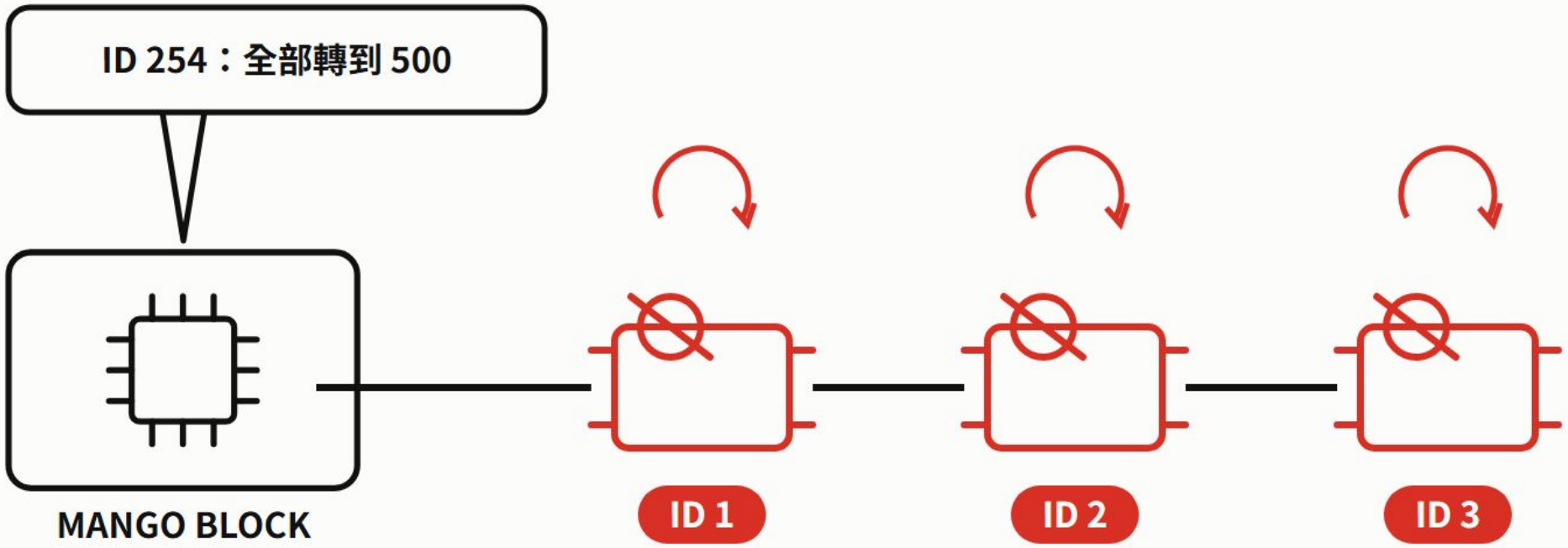





**兩顆同 ID → 同時回答 → 亂碼**

# 多舵機通訊






**ID 254 是廣播：全部都會做，但只能下指令**

# 多舵機通訊






**一起轉：連續送三個 move，三顆幾乎同時開始**

# 多顆一起轉

一步一步動起來（先 from servo import *）

```
move(2, 600, 1000)
```

只轉 ID 2

```
for i in (1, 2, 3): move(i, 600, 2000)
```

三顆一起轉到 600

```
for i in (1, 2, 3): move(i, 500, 2000)
```

三顆一起回中間

```
[pos(i) for i in (1, 2, 3)]
```

讀三顆的位置

# 寫自己的舵機程式

# 寫自己的程式



example.py 的輸出

```
id 1 starts at 1000
    went to 951
    back at 999
id 2 starts at 0
    went to 49
    back at 1
id 3 starts at 560
    went to 511
    back at 559
```

1

## 打開 example.py Run 他

這是個範例程式

響幾聲就是幾號在轉

每顆朝中間轉一點，再轉回來

# 寫自己的程式



play.py

```
from servo import *


# start your own code in
here
beep()
```

2

## 換你寫：play.py

用 move / pos / vin / beep
寫出你自己的程式

# 寫自己的程式

在 play.py 裡完成.....

1. 讓一顆舵機來回擺 3 次

2. 三顆同時轉

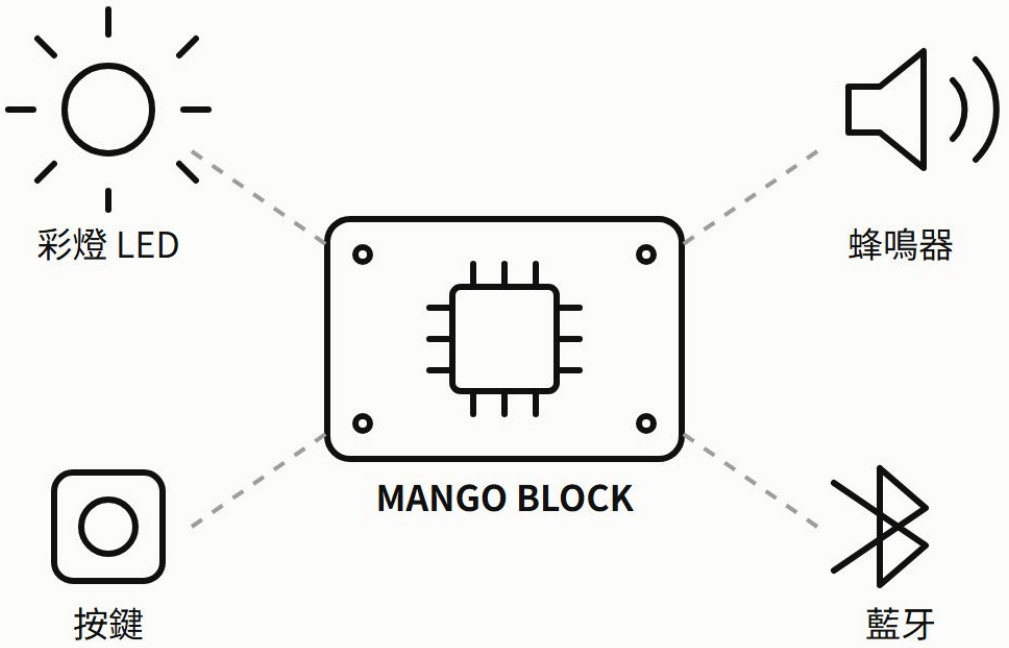
3. 透過按鈕控制 Servo

~ 自由發揮 ~

- 改接線之前
  先關外接電源
- 舵機固定好
  再做動作
- 小角度、慢速的轉

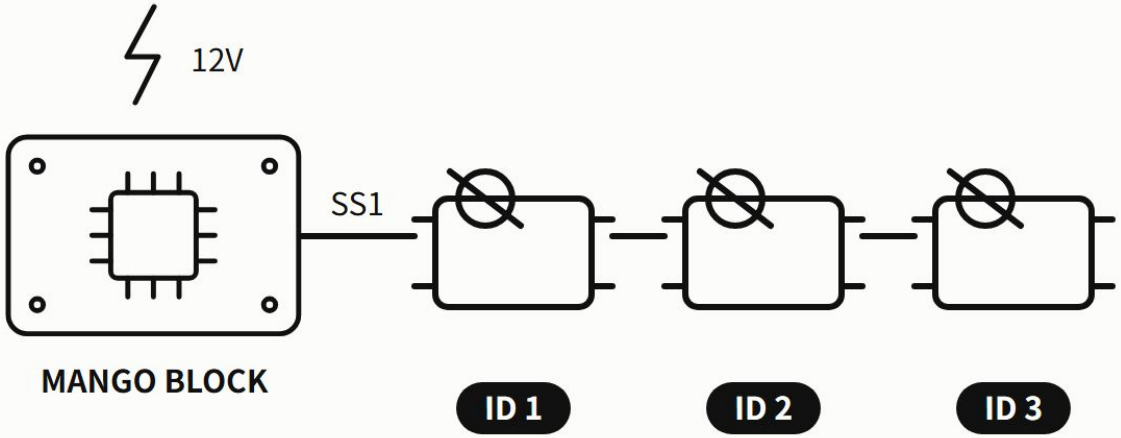
# 總結

今天學到.....




**如何使用 MANGO BLOCK**

**和其自備的配件功能**




**透過 MANGO BLOCK**

**操控舵機**

# Q & A

有任何問題都可以問喔
上午下午的都可以